COURS COMPLET

Formation AWS pour ingénieur DevOps

Du débutant à l'expert cloud : cours, démos, labos et préparation aux certifications

18 modules | 15 labos | Projet final | Certifications

Version imprimable du dépôt github.com/nzembouong18/aws.
Le code des démos et des labos se trouve dans le dépôt, à côté de chaque cours.

Sommaire

Formation AWS pour ingénieur DevOps — du débutant à l'expert

Module 00 — Démarrage : compte, sécurité, outils

Module 01 — Fondamentaux du cloud AWS

Module 02 — IAM : identités et accès

Module 03 — Réseau : Amazon VPC

Module 04 — Compute : Amazon EC2

Module 05 — Stockage : S3, EBS, EFS

Module 06 — Bases de données : RDS, Aurora, DynamoDB, ElastiCache

Module 07 — Haute disponibilité, élasticité et edge

Module 08 — Serverless et messaging

Module 09 — Conteneurs : ECR, ECS, Fargate, EKS

Module 10 — Infrastructure as Code : CloudFormation, CDK, Terraform

Module 11 — CI/CD sur AWS

Module 12 — Observabilité, audit et conformité

Module 13 — Sécurité avancée

Module 14 — Architecture, Well-Architected, résilience et FinOps

Module 15 — Multi-comptes et gouvernance

Module 16 — Projet final : plateforme cloud complète

Module 17 — Préparation aux certifications AWS

Questions d'entraînement (style examen)

Flashcards

Aide-mémoire pour l'examen

Suivi de progression et cahier d'erreurs

Formation AWS pour ingénieur DevOps — du débutant à l'expert

Formation complète, en français, pour **s'auto-former** ou **former des profils** (cours + démos + labos + projet final + préparation aux certifications AWS).

> Tout est « as code » : les labos utilisent l'AWS CLI et Terraform, versionnés dans ce dépôt.

À qui s'adresse cette formation ?

Profil	Point de départ	Objectif
Admin sys / dev junior	Module 00	Devenir DevOps AWS opérationnel
DevOps déjà à l'aise sur un autre cloud	Modules 02 → 10	Transposer ses acquis sur AWS
Ingénieur visant une certification	Module 17 + modules concernés	Réussir SAA / DVA / SOA / DOP

Prérequis : bases Linux (shell, SSH), réseau (IP, DNS, HTTP), Git. Pas de connaissance AWS requise.

Programme

#	Module	Durée	Niveau	Contenu clé
00	Démarrage	0,5 j	Débutant	Compte, sécurisation, budgets, CLI, outils
01	Fondamentaux du cloud AWS	0,5 j	Débutant	Régions, AZ, responsabilité partagée, tarification
02	IAM	1 j	Débutant → Interm.	Users, rôles, policies, STS, IAM Identity Center
03	Réseau VPC	1,5 j	Intermédiaire	Subnets, routage, NAT, SG/NACL, endpoints, peering, TGW
04	Compute EC2	1 j	Débutant → Interm.	AMI, types, user-data, SSM, spot, Launch Templates
05	Stockage	1 j	Débutant → Interm.	S3, EBS, EFS, cycle de vie, chiffrement
06	Bases de données	1 j	Intermédiaire	RDS, Aurora, DynamoDB, ElastiCache
07	Haute dispo & edge	1 j	Intermédiaire	ALB/NLB, Auto Scaling, Route 53, CloudFront

#	Module	Durée	Niveau	Contenu clé
08	Serverless & messaging	1,5 j	Intermédiaire	Lambda, API Gateway, SQS/SNS, EventBridge, Step Functions
09	Conteneurs	2 j	Interm. → Avancé	ECR, ECS/Fargate, EKS
10	Infrastructure as Code	2 j	Interm. → Avancé	CloudFormation, CDK, Terraform (modules, state, CI)
11	CI/CD	1,5 j	Intermédiaire	GitHub Actions + OIDC, CodePipeline, CodeBuild, CodeDeploy
12	Observabilité	1 j	Intermédiaire	CloudWatch, X-Ray, CloudTrail, Config, alerting
13	Sécurité	1,5 j	Avancé	KMS, Secrets Manager, WAF, GuardDuty, Security Hub
14	Architecture & FinOps	1 j	Avancé	Well-Architected, HA/DR, coûts
15	Multi-comptes & gouvernance	1 j	Expert	Organizations, SCP, Control Tower, landing zone
16	Projet final	3 j	Expert	Plateforme 3-tiers + CI/CD + observabilité
17	Préparation aux certifications	2 sem.	Tous	Plans de révision, questions, flashcards, stratégie d'examen

Total : environ 22 jours de formation pratique (≈ 8 à 12 semaines en autoformation à mi-temps).

Structure d'un module

Chaque dossier contient :

- `README.md` — le **cours** : objectifs, concepts, schémas, bonnes pratiques, erreurs fréquentes, résumé, quiz.
- `demos/` ou sections « Démo » — manipulations à **regarder/reproduire** guidées par le formateur.
- `labs/` — **labos** autonomes (énoncé, étapes, critères de validation, nettoyage) avec code de correction.

Parcours d'apprentissage

```
Débutant      00 → 01 → 02 → 03 (bases) → 04 → 05
Intermédiaire 03 → 06 → 07 → 08 → 12
Avancé        09 → 10 → 11 → 13 → 14
Expert        15 → 16 (projet final) → 17 (certifications)
```

Certifications visées :

Niveau	Certification
Fondation	Cloud Practitioner (CLF-C02)
Associate	Solutions Architect (SAA-C03), Developer (DVA-C02), CloudOps Engineer (SOA)
Professional	**DevOps Engineer Professional (DOP-C02)**, Solutions Architect Professional (SAP-C02)
Spécialité	Security (SCS), Advanced Networking (ANS-C01)

Vérifiez toujours les codes et versions d'examen sur aws.amazon.com/certification : AWS les renouvelle régulièrement.

⚠️ Coûts et sécurité — à lire avant de commencer

1. Créez un **compte dédié à la formation** (jamais de production).
2. Activez **MFA** sur le root et créez un **budget d'alerte** dès le module 00.
3. La plupart des labos tiennent dans le **Free Tier** ; les ressources payantes sont signalées par 💰 (NAT Gateway, EKS, ALB, RDS Multi-AZ…).
4. **Détruisez** systématiquement vos ressources en fin de labo (`terraform destroy` / section « Nettoyage »).
5. Ne commitez jamais de clés d'accès, `*.tfstate` ou fichiers `.pem`.

Comment utiliser ce dépôt

```
git clone https://github.com/nzembouong18/aws.git
cd aws
# Lire le cours
less 00-demarrage/README.md
# Faire un labo
cd 03-reseau-vpc/labs/lab-vpc
terraform init && terraform apply
```

Convention : région par défaut `eu-west-3` (Paris) — adaptable via la variable `region`.

Support de présentation

Un PowerPoint de 124 diapositives (avec notes de présentation) couvre toute la formation : `presentation/`. Il se régénère depuis `presentation/content.js`.

Méthode pédagogique recommandée (formateur)

- **Format de journée** : 30 % cours, 20 % démo, 50 % labo.
- Chaque module se termine par un **quiz** et un **critère de validation** (« je sais faire X sans aide »).
- Le **projet final** (module 16) sert d'évaluation de fin de parcours.
- Une rétrospective « quels coûts ai-je générés ? » en fin de journée ancre la culture FinOps.

Licence

Contenu à usage pédagogique. Les noms de services et certifications appartiennent à Amazon Web Services.

Module 00 — Démarrage : compte, sécurité, outils

Durée : 0,5 jour · **Niveau** : débutant · **Coût** : 0 €

Objectifs

- Créer et **sécuriser** un compte AWS de formation
- Mettre en place **budgets et alertes** de coûts
- Installer et configurer l'**AWS CLI v2**, Terraform et les outils annexes
- Comprendre comment on interagit avec AWS (Console, CLI, SDK, API, IaC)

1. Comment interagir avec AWS

Tout service AWS est une **API HTTPS signée** (SigV4). Tous les outils ne sont que des clients de cette API :

Outil	Usage	Quand
Console web	Découverte, debug visuel	Apprentissage, lecture
AWS CLI	Scripts, automatisation, exploration	Quotidien d'un DevOps
SDK (boto3, SDK JS/Go…)	Applications, Lambda	Code applicatif
IaC (Terraform, CloudFormation, CDK)	Infrastructure reproductible	**Seul mode accepté en production**
CloudShell	CLI dans le navigateur, déjà authentifiée	Dépannage rapide

Règle d'or DevOps : *la console sert à comprendre, le code sert à construire.*

2. Créer et sécuriser le compte

2.1 Création

1. Inscrivez-vous sur aws.amazon.com (e-mail dédié, ex. `aws-formation@votredomaine`).
2. Choisissez le plan gratuit/Free Tier selon l'offre en vigueur.

2.2 Sécurisation immédiate (checklist)

- ☐ **MFA sur l'utilisateur root** (application TOTP ou clé FIDO2)
- ☐ Ne **jamais** créer de clés d'accès pour root
- ☐ Créer un administrateur via **IAM Identity Center** (recommandé) ou un utilisateur IAM `admin` avec MFA

- [ ] Se connecter avec cet admin pour tout le reste ; ranger le root dans un coffre
- [ ] Renseigner les contacts de facturation/sécurité (Account settings)
- [ ] Activer l'accès aux **données de facturation** pour IAM

2.3 Budget d'alerte (obligatoire)

Console → *Billing and Cost Management* → *Budgets* → *Create budget* → *Cost budget* 10 € / mois, alertes à 50 %, 80 %, 100 % (réel) et 100 % (prévu).

Équivalent CLI : voir `demos/demo-01-budget.sh`.

3. Installer les outils

```
# AWS CLI v2 (Linux x86_64)
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o awscliv2.zip
unzip awscliv2.zip && sudo ./aws/install
aws --version

# Terraform (via tfenv recommandé pour gérer les versions)
git clone --depth=1 https://github.com/tfutils/tfenv.git ~/.tfenv
export PATH="$HOME/.tfenv/bin:$PATH"
tfenv install latest && tfenv use latest

# Outils utiles
sudo apt-get install -y jq git make     # jq = parsing JSON indispensable
pip install --user awscli-local checkov # checkov : scan sécurité IaC
```

Optionnels : `session-manager-plugin` (SSM), `kubectl`, `docker`, `aws-sam-cli`, `cdk` (npm).

4. Configurer l'authentification

Option recommandée : IAM Identity Center (SSO)

```
aws configure sso
# SSO session name : formation
# SSO start URL    : https://<id>.awsapps.com/start
# SSO region       : eu-west-3
aws sso login --profile formation
export AWS_PROFILE=formation
aws sts get-caller-identity
```

Option simple (compte de formation uniquement) : clés IAM

```
aws configure --profile formation   # Access key / Secret / region eu-west-3 / output json
```

> Des clés longue durée sont un risque : on les remplace par SSO ou par des rôles temporaires (voir module 02).

Fichiers

- `~/.aws/config` : profils, régions, SSO
- `~/.aws/credentials` : clés (à protéger, `chmod 600`)
- Variables : `AWS_PROFILE`, `AWS_REGION`, `AWS_ACCESS_KEY_ID`… (priorité sur les fichiers)

Ordre de résolution des credentials : **options CLI → variables d'env → profil → rôle de conteneur → rôle d'instance (IMDS)**.

5. Premières commandes

```
aws sts get-caller-identity                       # Qui suis-je ?
aws ec2 describe-regions --query 'Regions[].RegionName' --output text
aws s3 ls
aws ec2 describe-availability-zones --region eu-west-3 \
  --query 'AvailabilityZones[].[ZoneName,State]' --output table
```

Astuces CLI essentielles :

- `--query` (JMESPath) pour filtrer, `--output table|json|text|yaml`
- `--dry-run` sur les actions EC2 pour tester les permissions
- `aws <service> help`, `aws configure list`
- `--debug` pour comprendre une erreur d'auth

6. Démo

Démo 01 — Budget en CLI : `demos/demo-01-budget.sh`
Le formateur crée un budget de 10 € avec alerte e-mail, puis montre où le consulter dans la console.

Démo 02 — Tour de la console : barre de recherche, sélecteur de région, *Service Quotas*, *Cost Explorer*, *CloudShell*, *Tag Editor*.

7. Labo 00 — Environnement prêt à l'emploi

Énoncé : mettre en place votre poste et valider qu'il peut déployer sur AWS.

Étapes :

1. Compte sécurisé (checklist 2.2) + budget.
2. CLI + Terraform installés.
3. Profil `formation` configuré (SSO de préférence).
4. Exécuter le script de validation : `labs/lab-00/check-env.sh`.
5. Déployer et détruire un bucket S3 via Terraform : `labs/lab-00/terraform/`.

Validation : le script affiche ✅ `Environnement OK`, et `terraform destroy` supprime tout.

```
cd labs/lab-00 && ./check-env.sh
cd terraform && terraform init && terraform apply -auto-approve
terraform destroy -auto-approve
```

Erreurs fréquentes

Symptôme	Cause	Remède
`Unable to locate credentials`	Pas de profil / session SSO expirée	`aws sso login`, vérifier `AWS_PROFILE`
`ExpiredToken`	Credentials temporaires expirés	Se reconnecter
Ressource introuvable alors qu'elle existe	Mauvaise **région**	Vérifier `--region`, sélecteur console
`AccessDenied`	Policy manquante / SCP / boundary	Voir module 02

Quiz

1. Pourquoi ne pas utiliser le compte root au quotidien ?
2. Quelle est la différence entre une clé d'accès longue durée et un credential temporaire ?
3. Citez trois façons de fournir des credentials à la CLI et leur ordre de priorité.
4. Pourquoi créer un budget *avant* de lancer la moindre ressource ?

Réponses

1. Pouvoirs illimités, non restreignables par IAM, cible n°1 des attaquants ; on le réserve à quelques actions (fermer le compte, changer le support…).
2. Les temporaires (STS) expirent (15 min à 12 h) et limitent l'impact d'une fuite.
3. Options CLI > variables d'environnement > profil/fichiers > rôle de conteneur/instance.
4. Pour détecter tôt une dérive de coûts (ressource oubliée) ; AWS n'impose pas de plafond dur.

Pour aller plus loin

- Documentation : *AWS CLI User Guide*, *IAM Identity Center*
- Workshop : « AWS Well-Architected Labs — 100 Level Cost »

Module 01 — Fondamentaux du cloud AWS

Durée : 0,5 jour · **Niveau** : débutant · **Coût** : 0 €
Certification : cœur du Cloud Practitioner (CLF-C02)

Objectifs

- Expliquer le cloud computing (IaaS / PaaS / SaaS) et ses avantages
- Comprendre **Régions, Availability Zones, Edge Locations**
- Appliquer le **modèle de responsabilité partagée**
- Comprendre la **tarification** et choisir une région
- Situer les familles de services AWS

1. Le cloud computing

Caractéristiques (NIST) : libre-service à la demande, accès réseau large, mutualisation des ressources, élasticité rapide, service mesuré.

Modèle	Vous gérez	AWS gère	Exemples
IaaS	OS, runtime, app, données	Matériel, virtualisation	EC2, EBS, VPC
PaaS	App, données	OS, runtime, scaling	Elastic Beanstalk, RDS, Lambda, Fargate
SaaS	Usage	Tout	WorkMail, Amazon Connect

Avantages classiques : passer du **CapEx à l'OpEx**, élasticité, rapidité de déploiement, couverture mondiale en quelques minutes, économies d'échelle.

2. L'infrastructure globale

```
Partition AWS (aws, aws-cn, aws-us-gov)
 └── Région (ex. eu-west-3 Paris) — zone géographique isolée
      └── Availability Zone (eu-west-3a/b/c) — 1+ datacenters, liaisons à très faible latence
entre AZ
           └── Datacenters
 └── Edge Locations / Points of Presence — CloudFront, Route 53, Global Accelerator
 └── Local Zones, Wavelength, Outposts — extensions proches des utilisateurs/on-prem
```

Critères de choix d'une région : **conformité/souveraineté**, **latence**, **disponibilité des services**, **coût**.

Services selon leur portée :

- **Globaux** : IAM, Route 53, CloudFront, Organizations, WAF (scope CloudFront)
- **Régionaux** : EC2, S3 (bucket dans une région, nom global), VPC, RDS, Lambda…
- **Zonaux** : subnet, instance EC2, volume EBS

Principe de conception : concevoir pour la panne. Une AZ peut tomber → répartir sur ≥ 2 AZ (voir modules 03 et 07).

3. Responsabilité partagée

AWS : sécurité **DU** cloud	Vous : sécurité **DANS** le cloud
Datacenters, matériel, réseau physique	Données et chiffrement
Hyperviseur, services managés (plateforme)	IAM, MFA, politiques d'accès
Hôte de S3/DynamoDB/Lambda	OS invité, correctifs EC2, pare-feu (SG), config applicative

La répartition **varie selon le service** : plus il est managé, moins vous en gérez (RDS : AWS patche le moteur, vous gérez les utilisateurs et le schéma ; EC2 : vous patchez l'OS).

4. Tarification

Trois leviers principaux : **calcul**, **stockage**, **transfert sortant** (le trafic entrant est gratuit, le trafic inter-AZ et sortant Internet est facturé).

Modèle d'achat EC2	Réduction typique	Usage
On-Demand	0 %	Imprévisible, courte durée
Savings Plans / RI (1-3 ans)	jusqu'à ~70 %	Charge stable
Spot	jusqu'à ~90 %	Tolérant aux interruptions (batch, CI)
Dedicated Host	variable	Licences, conformité

Outils : **Pricing Calculator**, **Cost Explorer**, **Budgets**, **Cost and Usage Report**, **Trusted Advisor**.

Free Tier : trois types (essai, 12 mois, toujours gratuit) — vérifiez les conditions actuelles sur la page officielle.

5. Cartographie des services

Domaine	Services
Calcul	EC2, Lambda, ECS, EKS, Fargate, Batch, Elastic Beanstalk
Stockage	S3, EBS, EFS, FSx, Glacier, Storage Gateway
Base de données	RDS, Aurora, DynamoDB, ElastiCache, Redshift, DocumentDB, Neptune
Réseau	VPC, Route 53, CloudFront, ELB, API Gateway, Direct Connect, Transit Gateway
Sécurité	IAM, KMS, Secrets Manager, WAF, Shield, GuardDuty, Inspector, Security Hub

Domaine	Services
Intégration	SQS, SNS, EventBridge, Step Functions, MQ
Gestion/DevOps	CloudFormation, CloudWatch, CloudTrail, Config, Systems Manager, CodeBuild/Pipeline/Deploy
Analytique/ML	Athena, Glue, Kinesis, EMR, SageMaker, Bedrock

6. Les 6 piliers du Well-Architected (aperçu)

Excellence opérationnelle · Sécurité · Fiabilité · Performance · Optimisation des coûts · Durabilité. Développé au module 14.

7. Démos

Démo 01 — Explorer l'infrastructure globale : `demos/demo-01-global-infra.sh` liste régions, AZ et services disponibles dans une région.

Démo 02 — Calculer un coût : avec le *Pricing Calculator*, estimer 2 × `t3.small` + 100 Go EBS + 500 Go de S3 + 1 To de sortie à Paris, puis comparer avec Virginie (us-east-1).

8. Labo 01 — Cartographier son compte

Énoncé : produire un inventaire de votre compte et estimer le coût d'une architecture.

1. Lister les régions activées et choisir une région primaire et une de secours (justifier).
2. Pour la région primaire, lister les AZ et leurs IDs (`use1-az1` ≠ `us-east-1a` selon les comptes).
3. Avec `aws ssm get-parameters-by-path --path /aws/service/global-infrastructure/regions/eu-west-3/services`, lister les services disponibles à Paris.
4. Dans le Pricing Calculator, estimer le coût mensuel d'un site web : 2 EC2 t3.small, 1 ALB, 1 RDS db.t3.micro Multi-AZ, 50 Go S3, 200 Go de transfert sortant. Exporter et conserver le lien.
5. Rédiger 5 lignes : quelles parties relèvent de votre responsabilité sur cette architecture ?

Validation : fichier `inventaire.md` contenant régions, AZ, estimation chiffrée et responsabilités.

Quiz

1. Qu'est-ce qui distingue une Région d'une AZ ?
2. Vrai/faux : AWS patche l'OS de vos instances EC2.
3. Le transfert de données *entrant* depuis Internet est-il facturé ?
4. Quels services sont globaux : EC2, IAM, Route 53, S3 (bucket) ?
5. Quel modèle d'achat pour un job batch interruptible ?

Réponses

1. La région est une zone géographique contenant plusieurs AZ isolées ; une AZ est un ou plusieurs datacenters.
2. Faux, c'est votre responsabilité (sauf services managés).
3. Non, gratuit.
4. IAM et Route 53 (S3 : espace de noms global mais bucket régional).
5. Spot.

Module 02 — IAM : identités et accès

Durée : 1 jour · **Niveau** : débutant → intermédiaire · **Coût** : 0 €
Certifications : socle de TOUS les examens (SAA, DVA, SOA, DOP, SCS)

Objectifs

- Distinguer **utilisateurs, groupes, rôles, policies**
- Écrire et lire une policy JSON, comprendre l'**évaluation** des permissions
- Utiliser **STS / AssumeRole** et les credentials temporaires
- Appliquer le **moindre privilège**, les *permission boundaries* et les conditions
- Connaître **IAM Identity Center** et la fédération (OIDC pour CI/CD)

1. Concepts

Objet	Rôle
Utilisateur	Identité longue durée (humain ou programme). À éviter pour les humains → Identity Center
Groupe	Ensemble d'utilisateurs auquel on attache des policies
Rôle	Identité **assumable**, sans mot de passe, credentials temporaires. Pour services AWS, comptes tiers, fédération
Policy	Document JSON : *qui peut faire quoi sur quoi sous quelles conditions*
Principal	L'entité qui fait la requête (user, rôle, service, compte)

2. Anatomie d'une policy

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "LireLeBucketApp",
    "Effect": "Allow",
    "Action": ["s3:GetObject", "s3:ListBucket"],
    "Resource": [
      "arn:aws:s3:::mon-bucket",
      "arn:aws:s3:::mon-bucket/*"
    ],
    "Condition": {
      "IpAddress": {"aws:SourceIp": "203.0.113.0/24"},
      "Bool": {"aws:MultiFactorAuthPresent": "true"}
    }
  }]
}
```

- **Effect** : Allow / Deny
- **Action** : `service:Operation`, jokers autorisés (`s3:Get*`)

- **Resource** : ARN (`arn:partition:service:region:account-id:resource`)
- **Condition** : clés globales (`aws:RequestedRegion` , `aws:PrincipalTag/*` , `aws:SourceVpce` ...) et clés de service
- **Principal** (policies de ressource / de confiance uniquement)
- `NotAction` , `NotResource` : à manier avec précaution

Types de policies

Type	Attachée à	Remarque
Basée sur l'identité (managée AWS / client / inline)	user, groupe, rôle	Cas général
Basée sur la ressource	S3, SQS, KMS, Lambda, rôle (trust policy)...	Permet l'accès **cross-account** direct
Permissions boundary	user, rôle	Plafond de permissions
SCP	compte / OU (Organizations)	Plafond au niveau organisation
Session policy	session STS	Restreint encore la session
ACL	S3, VPC	Legacy

3. Logique d'évaluation

```
1. Deny explicite quelque part ?              → REFUSÉ
2. SCP (Organizations) autorise ?             → sinon REFUSÉ
3. Permission boundary autorise (si définie) ? → sinon REFUSÉ
4. Session policy autorise (si définie) ?     → sinon REFUSÉ
5. Allow dans une policy identité OU ressource → AUTORISÉ
6. Sinon                                      → REFUSÉ (implicite)
```

Règle de mémoire : **Deny explicite > Allow explicite > Deny implicite**. Pour un accès **même compte**, un Allow dans la policy de ressource *ou* d'identité suffit ; en **cross-account**, il faut un Allow des **deux côtés**.

4. Rôles et STS

```
Principal ──sts:AssumeRole──▶ Rôle (trust policy autorise le principal)
                          ◀── AccessKeyId + SecretKey + SessionToken (15 min–12 h)
```

Trust policy (qui peut assumer) ≠ **policy de permissions** (ce que le rôle peut faire).

Cas d'usage :

1. **Rôle de service** : EC2 (instance profile), Lambda (execution role), ECS (task role) → plus jamais de clés dans le code.
2. **Cross-account** : un compte « tooling » assume un rôle dans le compte « prod ».

3. **Fédération** : SAML/OIDC (GitHub Actions, EKS IRSA/Pod Identity, Google…).
4. **Break-glass** : rôle d'urgence audité.

```
aws sts assume-role --role-arn arn:aws:iam::123456789012:role/Deployer \
  --role-session-name ci --duration-seconds 3600
# External ID : protège contre le « confused deputy » pour les tiers
```

5. Bonnes pratiques

1. **MFA** partout, root verrouillé.
2. **Rôles > utilisateurs ; credentials temporaires > clés longues.**
3. **Moindre privilège** : partir de rien, élargir via les journaux (**IAM Access Analyzer** génère des policies depuis CloudTrail ; *last accessed* repère les droits inutilisés).
4. Utiliser des **conditions** (région, tags, IP, VPC endpoint, MFA).
5. **ABAC** (contrôle par tags) pour passer à l'échelle : `aws:PrincipalTag/team == aws:ResourceTag/team`.
6. Pas de `"Action":"*"` + `"Resource":"*"` hors compte bac à sable.
7. Rotation/suppression des clés inutilisées ; rapport d'identifiants (*credential report*).
8. Déléguer la création de rôles avec **permission boundary** obligatoire.

6. IAM Identity Center (ex-AWS SSO)

Point d'entrée unique pour les humains : annuaire intégré ou IdP externe (Entra ID, Okta), **permission sets** assignés à des groupes sur des comptes → crée des rôles IAM dans chaque compte. Sessions courtes, SSO CLI (`aws sso login`).

7. GitHub Actions → AWS sans clé (OIDC)

Le fournisseur d'identité `token.actions.githubusercontent.com` est déclaré dans IAM ; le rôle est assumable par `sts:AssumeRoleWithWebIdentity` avec conditions sur `sub` (repo/branche). Détaillé et mis en œuvre au **module 11**.

8. Démos

Démo 01 — Lire un refus

```
aws iam simulate-principal-policy \
  --policy-source-arn arn:aws:iam::<compte>:user/alice \
  --action-names s3:GetObject ec2:RunInstances \
  --resource-arns "*"
```

Puis décoder un message `AccessDenied` via `aws sts decode-authorization-message --encoded-message <...>`.

Démo 02 — AssumeRole en ligne de commande

`demos/demo-02-assume-role.sh`

Démo 03 — Access Analyzer

Console → IAM → *Access Analyzer* : créer un analyzer, lister les ressources partagées en externe, générer une policy depuis l'activité CloudTrail.

9. Labo 02 — Délégation sécurisée avec Terraform

Scénario : l'équipe `dev` doit pouvoir (1) lire et écrire dans un bucket applicatif, (2) lancer des instances **uniquement** en `eu-west-3` et seulement si elles portent le tag `team=dev`, avec MFA obligatoire pour assumer le rôle.

Code : `labs/lab-iam/`

Étapes :

1. `terraform init && terraform apply`
2. Examinez les policies créées (`main.tf`).
3. Assumez le rôle `dev-role` (output `assume_command`) en tant que `alice-test` : Terraform ne crée ni clé d'accès ni MFA, ajoutez-les à la main (console IAM → *Security credentials*), car la trust policy exige le MFA.
4. Tentez `aws ec2 run-instances --dry-run` en `us-east-1` → refus attendu ; en `eu-west-3` sans tag → refus ; avec tag → `DryRunOperation`.
5. Utilisez le **Policy Simulator** pour vérifier.
6. **Défi** : ajoutez une *permission boundary* empêchant la création de nouveaux utilisateurs IAM.
7. `terraform destroy`.

Validation : vous expliquez à voix haute pourquoi chaque appel est autorisé ou refusé, en citant l'étape d'évaluation concernée.

Erreurs fréquentes

Erreur	Pourquoi
Rôle créé mais « not authorized to perform sts:AssumeRole »	La **trust policy** ne cite pas le principal, ou l'appelant n'a pas `sts:AssumeRole`
S3 : Allow `s3:GetObject` mais refus	La Resource doit finir par `/*` pour les objets, le bucket sans `/*` pour `ListBucket`
Accès cross-account échoue	Il manque l'un des deux côtés (identité + ressource)
Policy trop large « pour que ça marche »	Utiliser Access Analyzer pour affiner
Clés commitées sur Git	Révoquer **immédiatement**, auditer CloudTrail

Quiz

1. Un Deny dans une SCP est-il contournable par un Allow dans une policy d'identité ?
2. Différence entre trust policy et permission policy ?
3. Pourquoi préférer un rôle à un utilisateur pour une application sur EC2 ?
4. Que fait une permission boundary : accorde-t-elle des droits ?
5. Quelle condition force MFA ?

Réponses

1. Non, le Deny (ou l'absence d'Allow) dans la SCP plafonne.
2. La trust policy dit qui peut assumer le rôle ; la permission policy dit ce que le rôle peut faire.
3. Credentials temporaires auto-renouvelés via IMDS, pas de secret à stocker.
4. Non, elle fixe un maximum ; il faut quand même un Allow.
5. `"Bool": {"aws:MultiFactorAuthPresent": "true"}`.

Module 03 — Réseau : Amazon VPC

Durée : 1,5 jour · **Niveau** : intermédiaire · **Coût** : 0 € (💰 NAT Gateway ≈ 0,05 $/h)
Certifications : SAA, SOA, DOP, ANS (très présent)

Objectifs

- Concevoir un **VPC multi-AZ** (plan d'adressage CIDR, subnets publics/privés)
- Maîtriser **tables de routage, IGW, NAT, Security Groups, NACL**
- Connecter privément les services AWS (**endpoints**) et d'autres réseaux (**peering, Transit Gateway, VPN**)
- Diagnostiquer avec **VPC Flow Logs** et **Reachability Analyzer**

1. Le VPC

Un **VPC** est un réseau virtuel privé et isolé, **régional**, défini par un ou plusieurs blocs CIDR (RFC 1918, de /16 à /28). Les **subnets** sont **zonaux** (une seule AZ).

Chaque subnet réserve **5 adresses** (réseau, routeur, DNS, futur, broadcast). Un /24 offre donc 251 adresses utilisables.

Plan d'adressage recommandé

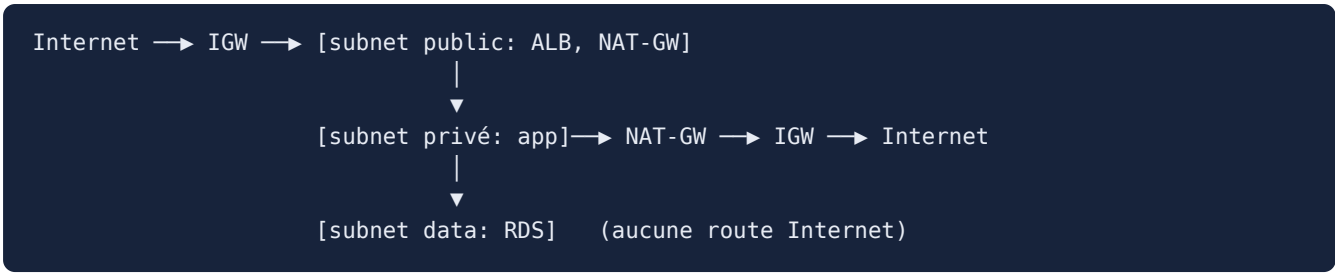
```
VPC 10.0.0.0/16                    (65 531 IP)
 ├─ public-a   10.0.0.0/24   AZ a   → ALB, NAT, bastion
 ├─ public-b   10.0.1.0/24   AZ b
 ├─ private-a  10.0.10.0/24  AZ a   → applications
 ├─ private-b  10.0.11.0/24  AZ b
 ├─ data-a     10.0.20.0/24  AZ a   → bases de données (aucune route Internet)
 └─ data-b     10.0.21.0/24  AZ b
```

Règle : **ne pas chevaucher** les CIDR des VPC/réseaux que vous voudrez relier (peering, VPN, TGW non transitifs sur overlap). Prévoyez large dès le départ.

2. Public vs privé

Un subnet est **public** s'il a une route `0.0.0.0/0 → Internet Gateway` ; ses instances ont aussi besoin d'une IP publique/Elastic IP.

Un subnet **privé** n'a pas de route directe vers l'IGW. Sa sortie Internet passe par une **NAT Gateway** (placée dans un subnet public, 1 par AZ pour la HA).



3. Composants

Composant	Rôle	Points d'attention
Route table	Route par subnet (association explicite) ; règle la plus spécifique gagne	Table `main` = défaut, ne pas l'utiliser pour du public
Internet Gateway	Entrée/sortie Internet, scalable, gratuite	1 par VPC
NAT Gateway	Sortie IPv4 pour le privé 💰	Par AZ, facturée à l'heure + Go ; alternative : NAT instance (moins fiable)
Egress-only IGW	Sortie IPv6	
Security Group	Pare-feu **stateful**, niveau ENI, **Allow uniquement**	Peut référencer un autre SG
NACL	Pare-feu **stateless**, niveau subnet, Allow + Deny, règles numérotées	Ouvrir les ports éphémères 1024-65535 en retour
VPC Endpoint – Gateway	S3 et DynamoDB, via table de routage, **gratuit**	
VPC Endpoint – Interface	PrivateLink : ENI + IP privée vers un service AWS/tiers 💰	DNS privé
Elastic IP	IP publique statique	Facturée si inutilisée

Security Group vs NACL

	Security Group	NACL
Niveau	Instance / ENI	Subnet
État	Stateful	Stateless
Règles	Allow seulement	Allow & Deny, ordre numérique
Défaut	Tout sortant, rien entrant	Tout autorisé (défaut)

Bonne pratique : chaîner les SG (`alb-sg` → `app-sg` → `db-sg`) en référençant les SG sources plutôt que des CIDR.

4. Connecter des réseaux

Besoin	Solution
2 VPC	**VPC Peering** (1-1, non transitif, pas d'overlap)
N VPC / on-prem (hub & spoke)	**Transit Gateway** (transitif, tables de routage, inter-régions)
On-prem chiffré via Internet	**Site-to-Site VPN** (2 tunnels IPsec)
On-prem lien dédié	**Direct Connect** (+ VPN pour chiffrer, ou MACsec)
Exposer un service à des centaines de VPC	**PrivateLink** (Endpoint Service + NLB)
Accès SSH/RDP sans port ouvert	**SSM Session Manager** (voir module 04)

5. DNS et IPv6

- Le DNS du VPC (`enableDnsSupport`, `enableDnsHostnames`) résout les noms internes ; le résolveur est à `base du VPC + 2`.
- **Route 53 Resolver** (endpoints entrants/sortants) pour le DNS hybride.
- IPv6 : blocs /56 attribués par AWS, adresses publiques par défaut → gérer avec SG et egress-only IGW.

6. Observabilité réseau

- **VPC Flow Logs** (ENI/subnet/VPC → CloudWatch Logs ou S3) : `ACCEPT` / `REJECT`, ports, volumes.
- **Reachability Analyzer** : analyse statique « A peut-il joindre B ? » avec le composant bloquant.
- **Traffic Mirroring**, **Network Access Analyzer**.

7. Démos

Démo 01 — Construire un VPC à la main (console) pour *comprendre* : VPC → subnets → IGW → route table → instance → échec/réussite de ping, puis l'équivalent CLI dans `demos/demo-01-vpc-cli.sh`.

Démo 02 — SG vs NACL : bloquer une IP avec une NACL (Deny) montre qu'un SG ne peut pas le faire ; constater l'effet *stateless* en retirant les ports éphémères.

Démo 03 — Reachability Analyzer : analyse entre une instance privée et Internet, repérer la route NAT manquante.

8. Labo 03 — VPC 3 niveaux avec Terraform

Énoncé : déployer un VPC 2 AZ avec subnets public/privé/data, IGW, NAT (optionnelle), SG chaînés, endpoint S3 gratuit, Flow Logs, puis valider les chemins réseau.

Code : `labs/lab-vpc/`

```
cd labs/lab-vpc
terraform init
terraform apply -var="enable_nat=false"        # 0 € : le privé n'a pas de sortie Internet
# Test : l'instance de test est atteignable via SSM, sans IP publique ni SSH
terraform apply -var="enable_nat=true"         # 💰 active la NAT (1 seule, mode économique)
terraform destroy
```

Questions guidées :

1. Pourquoi `curl https://example.com` échoue avec `enable_nat=false` mais `aws s3 ls` fonctionne (endpoint S3) ?
2. Quel est le nombre d'IP utilisables dans un /24 ? Pourquoi ?
3. Lisez les Flow Logs : retrouvez un `REJECT`.
4. Utilisez Reachability Analyzer : app privée → data privée sur 5432. Que manque-t-il ?
5. **Défi** : passez à une NAT par AZ (HA) et chiffrez le coût mensuel de la différence.

Validation : schéma annoté du VPC + capture d'un Flow Log `ACCEPT` et d'un `REJECT`.

Erreurs fréquentes

Symptôme	Cause
Instance publique inaccessible	Pas d'IP publique, route IGW absente, SG ou NACL
Instance privée sans Internet	Route vers NAT manquante ; NAT dans un subnet **privé** (doit être public)
Peering sans trafic	Routes non ajoutées des deux côtés / SG
Réponse bloquée avec NACL	Ports éphémères non ouverts (stateless)
Facture NAT élevée	Trafic vers S3/ECR via NAT → utiliser des endpoints

Quiz

1. Pourquoi un subnet ne peut-il pas s'étendre sur deux AZ ?
2. Quelle différence entre IGW et NAT Gateway ?
3. Le peering est-il transitif ?
4. Comment autoriser uniquement le trafic de l'ALB vers l'application ?
5. Quel endpoint est gratuit ?

Réponses

1. Un subnet est zonal par conception (isolation de panne).
2. L'IGW permet l'accès bidirectionnel Internet (IP publique requise) ; la NAT permet uniquement les connexions sortantes initiées depuis le privé.
3. Non ; utiliser Transit Gateway.
4. Règle entrante du SG de l'app avec comme source le SG de l'ALB.
5. Gateway endpoint (S3, DynamoDB).

Module 04 — Compute : Amazon EC2

Durée : 1 jour · **Niveau** : débutant → intermédiaire · **Coût** : Free Tier (`t3.micro` / `t2.micro` selon région)
Certifications : CLF, SAA, SOA, DVA

Objectifs

- Choisir le bon **type d'instance** et le bon **modèle d'achat**
- Créer, durcir et accéder à une instance (**SSM**, pas de SSH ouvert)
- Utiliser **AMI, user-data, Launch Templates, IMDSv2**
- Comprendre **EBS, snapshots, placement groups**

1. EC2 en bref

Machines virtuelles à la demande (virtualisation **Nitro**). Une instance = type + AMI + réseau (subnet, SG) + stockage + rôle IAM + clé/accès + user-data.

Familles d'instances

Famille	Optimisée pour	Exemple
T (t3, t4g)	Usage général, burstable (crédits CPU)	Dev, petits sites
M	Équilibre CPU/RAM	Applications
C	Calcul	Batch, encodage, CI
R / X	Mémoire	Bases, caches
I / D	Stockage local NVMe	Bases NoSQL
G / P / Inf / Trn	GPU / accélérateurs	ML/IA

Nomenclature : `m7g.xlarge` = famille **m**, génération **7**, processeur **g** (Graviton ARM, ~20 % moins cher à perf égale), taille **xlarge**.

2. Modèles d'achat

Modèle	Idéal
On-Demand	Court terme, imprévisible
Savings Plans (Compute / EC2 Instance)	Engagement $/h 1-3 ans, flexible
Reserved Instances	Engagement sur une configuration

Modèle	Idéal
Spot	Jusqu'à -90 %, interruption avec 2 min de préavis : CI, batch, workers sans état
Dedicated Hosts/Instances	Licences au socket, conformité
Capacity Reservations	Garantir la capacité en AZ

3. AMI, user-data, métadonnées

- **AMI** : image (OS + logiciels), régionale, copiable ; construisez vos « golden images » avec **EC2 Image Builder** ou **Packer**.
- **User-data** : script exécuté au 1er boot (cloud-init), limité à 16 Ko.
- **IMDS** (`http://169.254.169.254`) : métadonnées et credentials du rôle. **Imposer IMDSv2** (`http_tokens = required`) pour contrer le SSRF.

```
TOKEN=$(curl -sX PUT "http://169.254.169.254/latest/api/token" -H "X-aws-ec2-metadata-token-ttl-seconds: 60")
curl -s -H "X-aws-ec2-metadata-token: $TOKEN" http://169.254.169.254/latest/meta-data/instance-id
```

4. Accès : SSM plutôt que SSH

AWS Systems Manager Session Manager : shell dans le navigateur ou CLI, **sans port 22, sans bastion, sans clé**, journalisé (CloudTrail, S3/CloudWatch). Prérequis : agent SSM (présent sur Amazon Linux 2023), rôle `AmazonSSMManagedInstanceCore`, accès sortant HTTPS ou endpoints VPC.

```
aws ssm start-session --target i-0123456789abcdef0
aws ssm start-session --target i-0123 --document-name AWS-StartPortForwardingSession \
  --parameters '{"portNumber":["5432"],"localPortNumber":["5432"]}'
```

Autres capacités SSM : **Run Command**, **Patch Manager**, **State Manager**, **Parameter Store**, **Automation**, **Inventory**.

5. Stockage d'instance

Type	Caractéristique
EBS gp3	SSD par défaut, IOPS/débit réglables indépendamment de la taille
EBS io2	IOPS provisionnées, bases critiques
EBS st1 / sc1	HDD débit / froid
Instance store	NVMe local, éphémère (perdu à l'arrêt)

EBS est **zonal**, répliqué dans l'AZ ; **snapshots** incrémentaux dans S3 (régionaux, copiables, automatisables avec **Data Lifecycle Manager**). Chiffrement KMS recommandé (activez le chiffrement par défaut du compte/région).

6. Launch Templates

Modèle versionné d'une instance (AMI, type, SG, user-data, IAM profile, tags…) réutilisé par **Auto Scaling**, EC2 Fleet, Spot Fleet. Remplace les *Launch Configurations* (obsolètes).

7. Réseau d'instance

ENI (carte réseau virtuelle), IP privée fixe, IP publique (éphémère) ou **Elastic IP**. **Placement groups** : *cluster* (latence mini, une AZ), *spread* (instances sur matériels distincts), *partition* (gros clusters distribués).

8. Cycle de vie

`pending → running → stopping/stopped → terminated`. À l'arrêt : plus de facturation compute, mais EBS et EIP restent facturés. *Hibernate* sauvegarde la RAM. **Protection contre la résiliation** et **Stop protection** disponibles.

9. Démos

Démo 01 — Lancer une instance en CLI avec user-data + SSM : `demos/demo-01-run-instance.sh`
Démo 02 — IMDSv1 vs IMDSv2 : montrer qu'un `curl` simple échoue quand IMDSv2 est obligatoire.
Démo 03 — Spot : demander une instance Spot, comparer le prix (`aws ec2 describe-spot-price-history`).

10. Labo 04 — Serveur web durci avec Terraform

Énoncé : déployer un serveur nginx dans le VPC par défaut (ou celui du module 03), sans SSH, accessible via SSM, avec IMDSv2, volume chiffré gp3, et AMI toujours à jour.

Code : `labs/lab-ec2/`

Étapes :

1. `terraform apply`, ouvrir l'URL en sortie → page web.
2. `aws ssm start-session` → `systemctl status nginx`, `curl` IMDSv2.
3. Créer un **snapshot** du volume racine (`aws ec2 create-snapshot`), puis une **AMI** depuis l'instance.
4. Modifier le `user_data` : observer pourquoi l'instance est **remplacée** (immutabilité).
5. **Défi** : passez en `t4g.micro` (Graviton) ; que faut-il changer côté AMI ?
6. `terraform destroy` (supprimer aussi snapshot/AMI manuellement : hors state).

Validation : port 22 fermé, session SSM ouverte, `http_tokens=required` vérifié via `aws ec2 describe-instances` .

Erreurs fréquentes

Symptôme	Cause
Instance absente de SSM	Rôle manquant, pas de sortie HTTPS/endpoint, agent arrêté
`UnauthorizedOperation`	Permissions IAM/SCP (tester `--dry-run`)
`InsufficientInstanceCapacity`	Réessayer, changer d'AZ ou de type
Facture inattendue	Volumes EBS orphelins, EIP non attachées, snapshots
User-data non exécuté	Erreur dans le script : lire `/var/log/cloud-init-output.log`

Quiz

1. Pourquoi préférer Session Manager au SSH ?
2. Que devient l'instance store lors d'un arrêt ?
3. Quand choisir Spot ? Quand l'éviter ?
4. Quelle différence entre Launch Template et AMI ?
5. Que protège IMDSv2 ?

Réponses

1. Pas de port exposé ni de clés à gérer, accès via IAM, audit centralisé.
2. Les données sont perdues.
3. Charges tolérantes aux interruptions (batch, CI, stateless) ; à éviter pour bases de données/stateful critiques.
4. L'AMI est l'image disque ; le Launch Template est la configuration complète de lancement qui référence l'AMI.
5. Contre le vol de credentials via SSRF (jeton de session requis, saut de réseau limité).

Module 05 — Stockage : S3, EBS, EFS

Durée : 1 jour · **Niveau** : débutant → intermédiaire · **Coût** : ≈ 0 €
Certifications : CLF, SAA, SOA, DVA (S3 est omniprésent)

Objectifs

- Maîtriser **Amazon S3** : classes de stockage, versioning, cycle de vie, sécurité, événements
- Choisir entre **S3, EBS, EFS, FSx** selon le besoin
- Sécuriser les données (chiffrement, Block Public Access, politiques de bucket)
- Automatiser sauvegarde et réplication

1. Choisir son stockage

Service	Type	Accès	Cas d'usage
S3	Objet	HTTP/API, global	Fichiers, backups, data lake, sites statiques, artefacts
EBS	Bloc	1 instance (Multi-Attach io2 en option), 1 AZ	Disques d'OS, bases
EFS	Fichier NFS	Milliers d'instances, multi-AZ	Contenu partagé, CMS
FSx	Fichier managé (Windows, Lustre, NetApp, OpenZFS)	SMB/NFS/Lustre	HPC, Windows
Instance store	Bloc local	1 instance	Cache, temporaire
Storage Gateway / DataSync / Snow	Hybride / migration		

2. Amazon S3

Stockage objet **durable (11 neuf)**, quasi illimité, **cohérence forte** lecture-après-écriture. Un objet = clé + données (≤ 5 To) + métadonnées + version. Les « dossiers » ne sont que des préfixes.

Classes de stockage

Classe	Accès	Remarque
Standard	Fréquent	Défaut
Intelligent-Tiering	Inconnu/variable	Bascule auto, sans frais de récupération
Standard-IA / One Zone-IA	Rare, récupération en ms	Durée min 30 j ; One Zone = 1 AZ

Classe	Accès	Remarque
Glacier Instant Retrieval	Archive, ms	90 j min
Glacier Flexible Retrieval	Archive, min–h	90 j min
Glacier Deep Archive	Archive longue, ~12 h	180 j min, le moins cher

Fonctions clés

- **Versioning** : protège contre suppression/écrasement ; **MFA Delete** en option. Un « delete » ajoute un *delete marker*.
- **Lifecycle** : transitions entre classes, expiration, nettoyage des uploads multipart incomplets et des anciennes versions.
- **Réplication** CRR / SRR (versioning requis des deux côtés).
- **Object Lock** (WORM : Governance/Compliance) et **Legal Hold**.
- **S3 Event Notifications / EventBridge** → Lambda, SQS, SNS.
- **Presigned URLs** : accès temporaire sans credentials.
- **Multipart upload** (> 100 Mo recommandé), **Transfer Acceleration**, **S3 Select**, **Batch Operations**, **Inventory**, **Storage Lens**.
- **Static website hosting** (+ CloudFront + OAC pour HTTPS, voir module 07).
- **Access Points**, **Multi-Region Access Points**.

Sécurité S3 (essentiel)

1. **Block Public Access** activé au niveau du compte.
2. **Chiffrement** : SSE-S3 (défaut, AES-256), **SSE-KMS** (audit, contrôle de clé), SSE-C, côté client. Depuis 2023, le chiffrement SSE-S3 est appliqué à tous les nouveaux objets.
3. **Policy de bucket** : exiger TLS, restreindre par VPC endpoint, par rôle.
4. **Désactiver les ACL** (`BucketOwnerEnforced`).
5. **Journalisation** : server access logs, CloudTrail data events.
6. **Macie** pour détecter les données sensibles.

Exemple — refuser le trafic non chiffré :

```
{
  "Sid": "DenyInsecureTransport",
  "Effect": "Deny",
  "Principal": "*",
  "Action": "s3:*",
  "Resource": ["arn:aws:s3:::mon-bucket", "arn:aws:s3:::mon-bucket/*"],
  "Condition": {"Bool": {"aws:SecureTransport": "false"}}
}
```

CLI utile

```
aws s3 mb s3://mon-bucket-unique-123
aws s3 cp fichier.txt s3://mon-bucket-unique-123/dossier/
aws s3 sync ./site s3://mon-bucket-unique-123 --delete
aws s3 presign s3://mon-bucket-unique-123/fichier.txt --expires-in 300
aws s3api list-object-versions --bucket mon-bucket-unique-123
aws s3api put-bucket-versioning --bucket B --versioning-configuration Status=Enabled
```

3. EBS en détail

- Volumes **gp3** (3 000 IOPS / 125 Mo/s de base), **io2 Block Express** (jusqu'à 256 000 IOPS), **st1/sc1** HDD.
- **Snapshots** incrémentaux ; **Fast Snapshot Restore** ; **Recycle Bin** ; **EBS Snapshots Archive**.
- Redimensionnement à chaud (`modify-volume`, puis `growpart` + `resize2fs` / `xfs_growfs`).
- Chiffrement transparent KMS ; un snapshot d'un volume chiffré est chiffré.
- Sauvegarde centralisée : **AWS Backup** (plans, rétention, copie inter-région/compte, vault lock).

4. EFS

NFSv4.1 managé, **élastique**, multi-AZ, accès via *mount targets* (1 par AZ + SG sur le port 2049). Modes de performance (General Purpose / Max I/O), débit (Elastic / Provisioned), classes **Standard / IA** avec lifecycle. Linux uniquement (Windows → FSx).

```
sudo dnf install -y amazon-efs-utils
sudo mount -t efs -o tls fs-0123456789abcdef0:/ /mnt/efs
```

5. Sauvegarde et PRA (aperçu)

- **RPO** (perte de données acceptable) et **RTO** (temps de reprise) pilotent le choix. Détails au module 14.
- AWS Backup : politiques centralisées pour EBS, EFS, RDS, DynamoDB, S3…

6. Démos

Démo 01 — Cycle de vie d'un objet : `demos/demo-01-s3-versioning.sh` — versioning, suppression, restauration d'une version.
Démo 02 — Presigned URL : partager un fichier privé 5 minutes.
Démo 03 — Site statique sécurisé (voir module 07 pour la version CloudFront).
Démo 04 — Agrandir un volume EBS à chaud.

7. Labo 05 — Bucket de production en Terraform

Énoncé : créer un bucket « données applicatives » conforme aux bonnes pratiques :

- chiffrement SSE-KMS avec clé dédiée et *bucket key*
- versioning + règle de cycle de vie (Standard → IA à 30 j → Glacier IR à 90 j, expiration des anciennes versions à 180 j)
- Block Public Access, ACL désactivées, TLS obligatoire
- notification d'événement vers SQS lors d'un `ObjectCreated`

Code : `labs/lab-s3/`

Étapes :

1. `terraform apply`, envoyer un fichier, lire le message dans la file SQS (`aws sqs receive-message`).
2. Écraser le fichier 3 fois, lister les versions, restaurer la première.
3. Tenter un accès HTTP (non TLS) : `curl http://bucket.s3.amazonaws.com/...` → refus.
4. **Défi** : ajouter la réplication vers un second bucket d'une autre région.
5. `terraform destroy`.

Validation : vous prouvez par commande que (a) l'accès public est bloqué, (b) 3 versions existent, (c) l'événement est reçu.

Erreurs fréquentes

Problème	Explication
`AccessDenied` malgré une policy	Chiffrement KMS : il faut aussi `kms:Decrypt` sur la clé
Données publiques par erreur	Ne pas activer Block Public Access ; ACL anciennes
Nom de bucket refusé	Unique globalement, 3-63 caractères, minuscules
Perte de données EBS	Instance store ou volume `DeleteOnTermination=true` sans snapshot

Quiz

1. Quelle classe S3 pour des données dont le profil d'accès est inconnu ?
2. Comment annuler la suppression d'un objet versionné ?
3. EBS peut-il être monté depuis deux AZ ?
4. Pourquoi `s3:ListBucket` et `s3:GetObject` ont-ils des ARN différents ?
5. Quel service gère les sauvegardes centralisées ?

Réponses

1. Intelligent-Tiering.
2. Supprimer le *delete marker* (ou copier une version précédente).
3. Non, un volume EBS est lié à une AZ (utiliser un snapshot pour migrer).
4. `ListBucket` agit sur le bucket, `GetObject` sur les objets (`/*`).
5. AWS Backup.

Module 06 — Bases de données : RDS, Aurora, DynamoDB, ElastiCache

Durée : 1 jour · **Niveau** : intermédiaire · **Coût** : 💰 RDS/Aurora (Free Tier limité), DynamoDB on-demand ≈ 0 €
Certifications : SAA, DVA, SOA, DOP

Objectifs

- Choisir la bonne base selon le besoin (relationnel, clé-valeur, cache, analytique)
- Déployer **RDS Multi-AZ**, répliques de lecture, sauvegardes, restauration PITR
- Modéliser et interroger **DynamoDB** (clés, index, capacité, TTL, streams)
- Sécuriser (réseau privé, KMS, Secrets Manager, IAM auth)

1. Quelle base choisir ?

Besoin	Service
SQL classique (MySQL, PostgreSQL, MariaDB, Oracle, SQL Server)	**RDS**
SQL cloud-native, haute perf, 6 copies sur 3 AZ	**Aurora** (MySQL/PostgreSQL), **Aurora Serverless v2**
Clé-valeur / documents, latence ms, échelle illimitée	**DynamoDB**
Cache en mémoire (Redis/Valkey, Memcached)	**ElastiCache** / MemoryDB
Entrepôt de données	**Redshift**
Documents Mongo-compatible	DocumentDB
Graphe / série temporelle / ledger	Neptune / Timestream / QLDB (remplacé)
Analyse sur S3	Athena

2. RDS

Base managée : AWS gère matériel, OS, installation, patches du moteur, sauvegardes, bascule. Vous gérez schéma, requêtes, paramètres, utilisateurs.

Haute disponibilité et scalabilité

Fonction	Objectif	Détail
Multi-AZ (instance)	**Disponibilité / DR**	Réplication **synchrone** vers un standby d'une autre AZ ; bascule auto (1-2 min) ; *le standby ne sert pas les lectures*

Fonction	Objectif	Détail
Multi-AZ cluster	HA + lectures	2 standbys lisibles
Read Replicas	**Performance (lecture)**	Réplication **asynchrone**, jusqu'à 15, inter-régions possibles, promouvables
Sauvegardes auto	Restauration PITR	Rétention 1-35 j, restauration vers une **nouvelle** instance
Snapshots manuels	Archivage	Copie inter-région/compte
RDS Proxy	Pool de connexions	Indispensable avec Lambda ; réduit le temps de bascule

Mnémo : **Multi-AZ = disponibilité ; Read Replica = performance.**

Sécurité

- Subnet group **privé** (subnets « data »), SG n'autorisant que l'application.
- Chiffrement au repos (KMS, à l'**activation à la création**), TLS en transit.
- Identifiants dans **Secrets Manager** avec rotation ; **IAM database authentication** (jetons de 15 min).
- Protection contre la suppression, snapshot final.

3. Aurora

Stockage distribué partagé (6 copies sur 3 AZ, 10 → 128 To auto), jusqu'à 15 réplicas de lecture avec failover en < 30 s. Endpoints : **cluster** (écriture), **reader** (lecture répartie), **custom**. **Serverless v2** : capacité en ACU, scale fin ; **Global Database** : réplication inter-régions < 1 s, RPO ~1 s ; **Backtrack** (MySQL), clonage rapide.

4. DynamoDB

Base NoSQL serverless, latence à un chiffre en ms, multi-AZ native.

Modèle

- **Table** → **items** (≤ 400 Ko) → **attributs**.
- Clé primaire : **partition key** seule, ou **partition key + sort key**. La partition key doit répartir uniformément (éviter les « hot keys »).
- **GSI** (autre PK/SK, éventuellement cohérence éventuelle, capacité propre) / **LSI** (même PK, créé à la création).
- Concevoir **à partir des patterns d'accès**, pas du schéma : *single-table design*.

Capacité

Mode	Quand
On-demand	Trafic imprévisible, démarrage
Provisioned (+ Auto Scaling)	Trafic prévisible, optimisation du coût

1 RCU = 1 lecture fortement cohérente de 4 Ko/s (2 si éventuelle). 1 WCU = 1 écriture de 1 Ko/s.

Fonctions

Lectures **éventuellement cohérentes** (défaut) ou **fortement cohérentes** ; **transactions** ; **TTL** (expiration gratuite) ; **Streams** (→ Lambda, CDC) ; **Global Tables** (multi-région actif-actif) ; **PITR** (35 j) ; **DAX** (cache microseconde) ; export vers S3.

```
aws dynamodb put-item --table-name Commandes --item '{
  "client":{"S":"c42"}, "commande":{"S":"2026-001"}, "total":{"N":"59.90"}}'
aws dynamodb query --table-name Commandes \
  --key-condition-expression "client = :c AND begins_with(commande, :p)" \
  --expression-attribute-values '{":c":{"S":"c42"},":p":{"S":"2026"}}'
```

`Scan` parcourt tout : à proscrire en production sur grandes tables. Préférez `Query`.

5. ElastiCache

Redis/Valkey (persistance, réplication, Multi-AZ, clustering, pub/sub) ou Memcached (simple, multithread). Stratégies : **lazy loading** (cache-aside) + **write-through** ; toujours définir un **TTL**. Cas d'usage : sessions, cache de requêtes, classements, rate limiting.

6. Migration : DMS et SCT

DMS migre/réplique (full load + CDC) avec peu d'interruption ; **SCT** convertit les schémas pour migrations hétérogènes (Oracle → PostgreSQL).

7. Démos

Démo 01 — RDS PostgreSQL Multi-AZ : créer l'instance, forcer un **failover** (`aws rds reboot-db-instance --force-failover`) et observer la bascule du CNAME.
Démo 02 — Restauration PITR : supprimer des lignes puis restaurer à T-5 min vers une nouvelle instance.
Démo 03 — DynamoDB : `demos/demo-01-dynamodb.sh` (clé composite, Query, GSI, TTL).

8. Labo 06 — Base relationnelle sécurisée + table DynamoDB

Énoncé : déployer en Terraform (a) une instance RDS PostgreSQL dans des subnets privés avec mot de passe géré par Secrets Manager, chiffrée, sauvegardée, (b) une table DynamoDB on-demand avec GSI et TTL.

Code : `labs/lab-db/` — utilise le VPC par défaut (la base reste privée : `publicly_accessible=false`). Pour la connexion, lancez une instance SSM dans le même VPC (module 04).

Étapes :

1. `terraform apply -var="multi_az=false"` (économique) puis comparez avec `true`.
2. Récupérez le secret (`aws secretsmanager get-secret-value`) et connectez-vous via un hôte SSM (port forwarding).
3. Créez une table, insérez des données, créez un snapshot, restaurez-le.
4. Dans DynamoDB, insérez des items avec TTL ; vérifiez la suppression automatique.
5. **Défi** : ajoutez un Read Replica et mesurez le *replica lag* (CloudWatch `ReplicaLag`).
6. 💰 `terraform destroy` **immédiatement** après les tests.

Validation : la base n'est pas publique (`PubliclyAccessible=false`), chiffrée, et le mot de passe n'apparaît nulle part dans le code.

Erreurs fréquentes

Problème	Cause / solution
Timeout de connexion	SG, subnet group, ou DB dans le mauvais VPC
Lambda épuise les connexions	Utiliser RDS Proxy
Chiffrement impossible après coup	Restaurer un snapshot chiffré (copie avec KMS)
DynamoDB throttling	Hot partition ou capacité insuffisante
Coûts élevés	Instances oubliées, Multi-AZ en dev, snapshots conservés

Quiz

1. Multi-AZ ou Read Replica pour réduire la charge de lecture ?
2. Comment se connecter à RDS sans mot de passe statique ?
3. Pourquoi éviter `Scan` ?
4. Quelle option pour du multi-région actif-actif ?
5. Quel service pour limiter les connexions depuis des milliers de Lambdas ?

Réponses

1. Read Replica (Multi-AZ = HA).
2. IAM database authentication, ou secret géré/rotatif dans Secrets Manager.
3. Lit toute la table, coûteux et lent ; utiliser Query/GSI.
4. DynamoDB Global Tables (ou Aurora Global Database pour du relationnel avec écriture dans une seule région).
5. RDS Proxy.

Module 07 — Haute disponibilité, élasticité et edge

Durée : 1 jour · **Niveau** : intermédiaire · **Coût** : 💰 ALB ≈ 0,025 $/h + instances
Certifications : SAA (cœur), SOA, DOP

Objectifs

- Répartir la charge avec **ELB (ALB/NLB/GWLB)**
- Faire passer l'application à l'échelle avec **Auto Scaling**
- Gérer le DNS et le basculement avec **Route 53**
- Accélérer et protéger la livraison avec **CloudFront** (+ WAF, Shield)

1. Elastic Load Balancing

Type	Couche	Usage
ALB	7 (HTTP/HTTPS/gRPC/WebSocket)	Routage par chemin/hôte/header, cibles instances/IP/Lambda/ECS, auth OIDC/Cognito, WAF
NLB	4 (TCP/UDP/TLS)	Ultra faible latence, IP statique par AZ, PrivateLink
GWLB	3	Appliances réseau (pare-feu, IDS)
CLB	4/7	Obsolète

Notions : **listeners** (port + protocole + règles), **target groups** (cibles + **health checks**), **cross-zone load balancing**, **deregistration delay**, **sticky sessions**, terminaison TLS avec certificat **ACM** (gratuit), redirection HTTP→HTTPS, en-têtes `X-Forwarded-For/Proto`.

Schéma classique :

```
Internet → ALB (subnets publics, 2 AZ) → Target Group → ASG (subnets privés, 2 AZ) → RDS Multi-AZ
```

2. Auto Scaling Group (ASG)

Maintient un nombre d'instances entre **min / désiré / max**, réparties sur plusieurs AZ, à partir d'un **Launch Template**.

Politique	Principe
Target tracking (recommandé)	Garder une métrique cible (CPU 50 %, `RequestCountPerTarget`)
Step scaling	Paliers selon alarme CloudWatch

Politique	Principe
Scheduled	Pics prévisibles
Predictive	Prévision par ML

Autres éléments : **health check ELB** (remplace les instances jugées non saines), **cooldown/warm-up**, **lifecycle hooks**, **instance refresh** (déploiement progressif d'une nouvelle AMI), **mixed instances** (On-Demand + Spot), **warm pools**.

Principe d'architecture : **instances sans état, jetables** ; sessions dans ElastiCache/DynamoDB, fichiers dans S3.

3. Route 53

DNS managé (SLA 100 %) + enregistrement de domaines + health checks.

Enregistrement	Remarque
A / AAAA / CNAME / TXT / MX	Classiques
Alias	Spécifique AWS : pointe vers ALB, CloudFront, S3, API GW… **autorisé à l'apex** (exa `mple.com`), gratuit, sans requête facturée

Politiques de routage

Simple · **Pondéré** (canary, A/B) · **Latence** · **Failover** (actif/passif avec health check) · **Géolocalisation** · Géoproximité · Multi-valeurs · Basé sur IP.

DNS privé : **zones hébergées privées** associées à un VPC.

4. CloudFront (CDN)

~600 points de présence. Origines : S3, ALB, API Gateway, tout serveur HTTP.

- **Comportements** (par chemin), **politiques de cache**, **TTL**, **invalidations**.
- **OAC (Origin Access Control)** : S3 privé, accessible uniquement via CloudFront.
- **HTTPS** avec certificat **ACM en us-east-1**.
- **Signed URLs / cookies** pour du contenu privé.
- **CloudFront Functions / Lambda@Edge** : logique en bordure (réécriture, auth).
- **WAF** + **Shield** (Standard gratuit, Advanced payant) contre les attaques L7/DDoS.
- **Global Accelerator** : IP anycast statiques, bascule rapide, pour TCP/UDP non cachable.

5. Haute disponibilité : règles d'or

1. ≥ 2 AZ pour chaque tier.
2. Health checks à tous les niveaux, suppression automatique des instances défaillantes.

3. Pas d'état local ; pas de « pets ».
4. Données répliquées (Multi-AZ, S3, DynamoDB).
5. Tester la panne (Fault Injection Service).

6. Démos

Démo 01 — ALB + 2 instances : tuer une instance, observer la bascule (health check) et la recréation par l'ASG.
Démo 02 — Target tracking : charge CPU avec `stress-ng` (ou `hey`) et observer le scale-out dans l'historique d'activité ASG.
Démo 03 — Failover Route 53 : `demos/demo-01-route53-failover.md`.
Démo 04 — CloudFront + S3 privé (OAC) : voir labo.

7. Labo 07 — Application web hautement disponible

Énoncé : déployer, via Terraform, un ALB public + ASG (2 → 4 instances, 2 AZ) servant une page indiquant l'AZ, avec scaling par `ALBRequestCountPerTarget`, puis une distribution CloudFront devant l'ALB.

Code : `labs/lab-ha-web/`

```
cd labs/lab-ha-web
terraform init && terraform apply
curl $(terraform output -raw alb_url)          # rafraîchir : l'AZ alterne
```

Étapes :

1. Vérifier que les requêtes alternent entre 2 AZ.
2. Terminer une instance : mesurer le temps de remédiation.
3. Générer de la charge (`hey -z 3m -c 50 <url>`) → observer le scale-out.
4. **Instance refresh** : changer le `user_data` (nouvelle version de la page) et lancer un refresh sans interruption.
5. Comparer les en-têtes via CloudFront (`x-cache: Hit from cloudfront`).
6. `terraform destroy`.

Validation : zéro erreur 5xx pendant la terminaison d'une instance (test avec une boucle `curl`).

Erreurs fréquentes

Problème	Cause
Cibles `unhealthy`	SG cible n'autorise pas l'ALB, mauvais chemin de health check, appli non démarrée
Scale-in qui coupe des requêtes	Deregistration delay trop court / pas de graceful shutdown

Problème	Cause
Certificat CloudFront introuvable	ACM doit être en **us-east-1**
CNAME impossible à l'apex	Utiliser un enregistrement Alias
Instances recréées en boucle	Health check ELB trop strict ou grace period trop courte

Quiz

1. ALB ou NLB pour du routage par chemin ?
2. Quelle métrique cible pour scaler selon le trafic HTTP ?
3. Différence entre Alias et CNAME ?
4. Pourquoi mettre CloudFront devant S3 plutôt que d'exposer le bucket ?
5. Quel service fournit des IP anycast statiques ?

Réponses

1. ALB.
2. `ALBRequestCountPerTarget` (target tracking).
3. L'Alias fonctionne à l'apex, est natif AWS, gratuit et suit les changements d'IP ; le CNAME est standard mais impossible à l'apex.
4. HTTPS/domaine personnalisé, cache, WAF, bucket privé via OAC.
5. Global Accelerator.

Module 08 — Serverless et messaging

Durée : 1,5 jour · **Niveau** : intermédiaire · **Coût** : ≈ 0 € (Free Tier généreux)
Certifications : DVA (cœur), SAA, DOP

Objectifs

- Construire une API **serverless** (API Gateway + Lambda + DynamoDB)
- Découpler avec **SQS, SNS, EventBridge**
- Orchestrer avec **Step Functions**
- Connaître limites, cold start, concurrence, idempotence, DLQ

1. Lambda

Code exécuté à la demande, facturé à la ms × mémoire. Runtimes : Python, Node.js, Java, .NET, Go, Ruby, ou **image conteneur** (≤ 10 Go).

Paramètres clés

Paramètre	Valeur / remarque
Mémoire	128 Mo – 10 Go ; **CPU proportionnel** à la mémoire
Timeout	max **15 min**
/tmp	512 Mo – 10 Go
Concurrence	1 000 par région (défaut, relevable) ; **réservée** (plafond+garantie) ; **provisionnée** (pas de cold start)
Package	50 Mo zip / 250 Mo décompressé ; **layers**
Architecture	x86_64 ou **arm64** (moins cher)

Modèles d'invocation

- **Synchrone** : API Gateway, ALB, SDK → le client attend.
- **Asynchrone** : S3, SNS, EventBridge → file interne, 2 réessais, **DLQ / destinations**.
- **Polling (event source mapping)** : SQS, Kinesis, DynamoDB Streams → Lambda lit par lots ; gérer `batchItemFailures`.

Cold start

Initialisation du conteneur + du runtime + de votre code. Réduire : paquet léger, init hors handler (connexions réutilisées), arm64, SnapStart (Java), concurrence provisionnée.

Handler Python minimal

```
import json, os, boto3
table = boto3.resource("dynamodb").Table(os.environ["TABLE"])   # initialisé 1 fois par
environnement

def handler(event, context):
    item = json.loads(event["body"])
    table.put_item(Item=item)
    return {"statusCode": 201, "body": json.dumps({"ok": True})}
```

Bonnes pratiques

1. **Rôle d'exécution** au moindre privilège, une fonction = un rôle.
2. **Idempotence** (les événements peuvent être livrés plusieurs fois) → clés d'idempotence, *Powertools for AWS Lambda*.
3. Variables d'environnement pour la config ; secrets via Secrets Manager/Parameter Store (cache).
4. Logs structurés JSON, X-Ray.
5. Lambda en VPC seulement si nécessaire (accès RDS) → RDS Proxy.
6. Versions + **alias** (`prod`) + déploiements progressifs (CodeDeploy canary).

2. API Gateway

Type	Usage
HTTP API	Moins cher/rapide, JWT, CORS ; cas courant
REST API	Clés d'API, plans d'usage, validation, cache, WAF, transformations
WebSocket API	Temps réel

Autorisation : IAM, **Cognito**, **JWT**, Lambda authorizer. Throttling, stages, domaines personnalisés. Alternative : **Lambda Function URL** (HTTPS direct, sans API GW).

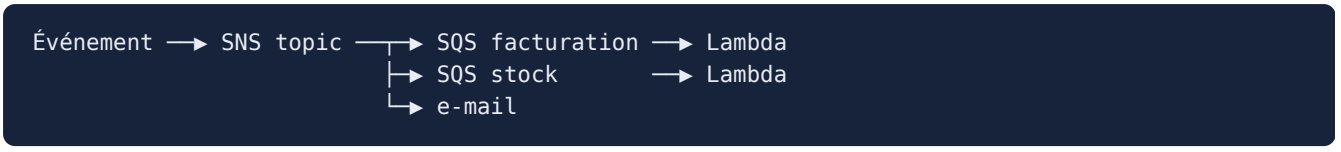
3. Messaging : SQS, SNS, EventBridge

Service	Modèle	Points clés
SQS Standard	File, pull	Débit illimité, *at-least-once*, ordre non garanti
SQS FIFO	File ordonnée	Exactly-once processing, groupes de messages, 300 msg/s (3 000 en batch) ; nom `.fifo`
SNS	Pub/Sub, push	Topics, abonnés SQS/Lambda/HTTP/e-mail/SMS, **filtrage** par attributs
EventBridge	Bus d'événements	Règles à motifs, sources AWS + SaaS + custom, **Scheduler**, Pipes, archivage/rejeu

Service	Modèle	Points clés
Kinesis Data Streams	Flux ordonné partitionné	Rejeu, plusieurs consommateurs, temps réel
Amazon MQ	Brokers ActiveMQ/RabbitMQ	Migration d'applis existantes

Paramètres SQS essentiels : **visibility timeout** (> durée de traitement), **long polling** (`WaitTimeSeconds=20`), **DLQ** (`maxReceiveCount`), délai, rétention (≤ 14 j), taille ≤ 256 Ko.

Patron « fan-out » : SNS → N files SQS (chaque consommateur à son rythme).



4. Step Functions

Orchestration visuelle par machine à états (ASL JSON) : `Task`, `Choice`, `Parallel`, `Map`, `Wait`, `Retry/Catch`. **Standard** (≤ 1 an, exactly-once, auditable) vs **Express** (≤ 5 min, volume élevé, at-least-once). Intégrations SDK directes avec 200+ services (sans Lambda « colle »).

5. Autres briques serverless

S3 · DynamoDB · **Cognito** (authentification utilisateurs) · **AppSync** (GraphQL) · **Fargate** (conteneurs sans serveur, module 09) · **Athena** · **Kinesis Firehose** · **AWS SAM** (framework IaC serverless).

6. Démos

Démo 01 — Lambda en CLI : `demos/demo-01-lambda.sh` (création, invocation, logs).
Démo 02 — SQS + DLQ : envoyer un message « empoisonné », constater 3 essais puis arrivée en DLQ.
Démo 03 — Step Functions : workflow `Validate → Choice → (Process | Reject)` avec Retry.

7. Labo 08 — API de gestion de tâches serverless

Énoncé : API REST `POST /tasks`, `GET /tasks/{id}` (HTTP API → Lambda → DynamoDB). Chaque tâche créée publie un événement sur SNS qui alimente une file SQS traitée par une seconde Lambda, avec DLQ.

Code : `labs/lab-serverless/`

```
cd labs/lab-serverless
terraform init && terraform apply
API=$(terraform output -raw api_url)
curl -X POST $API/tasks -H 'content-type: application/json' -d '{"title":"Apprendre AWS"}'
curl $API/tasks/<id>
aws logs tail /aws/lambda/lab08-worker --follow
```

Étapes :

1. Créer une tâche et vérifier la trace complète dans les logs du worker.
2. Envoyer un message invalide (`{"fail":true}` via SNS) → vérifier la DLQ après 3 échecs.
3. Mesurer le cold start (`REPORT … Init Duration`).
4. **Défi** : ajouter une règle EventBridge Scheduler qui nettoie les tâches terminées chaque nuit.
5. `terraform destroy` .

Validation : vous expliquez le chemin d'un message (sync puis async) et où il peut échouer.

Erreurs fréquentes

Problème	Cause
Messages SQS retraités en boucle	Visibility timeout < durée de traitement
Lambda timeout sur RDS	Lambda en VPC sans route/SG, pas de pool de connexions
Facture Lambda inattendue	Boucle récursive S3 → Lambda → S3
502 API Gateway	Mauvais format de réponse du handler
Doublons	Pas d'idempotence (at-least-once)

Quiz

1. Quelle durée maximale pour une Lambda ?
2. SNS ou SQS pour un fan-out ? Pourquoi les combiner ?
3. Que garantit une file FIFO ?
4. À quoi sert une DLQ ?
5. Comment éviter le cold start d'un endpoint critique ?

Réponses

1. 15 minutes.
2. SNS diffuse ; SQS bufferise et fiabilise chaque consommateur ; on combine : SNS → plusieurs SQS.
3. Ordre (par groupe de messages) et dé-duplication.
4. Isoler les messages échoués pour analyse sans bloquer le flux.
5. Provisioned concurrency / SnapStart / paquet léger.

Module 09 — Conteneurs : ECR, ECS, Fargate, EKS

Durée : 2 jours · **Niveau** : intermédiaire → avancé · **Coût** : 💰 Fargate ≈ quelques centimes/h ; EKS 0,10 $/h le plan de contrôle
Certifications : DOP (cœur), SAA, DVA, SOA

Objectifs

- Construire, signer et stocker des images dans **ECR**
- Déployer des services sur **ECS/Fargate** derrière un ALB, avec autoscaling
- Comprendre **EKS** : architecture, nœuds managés, Fargate profiles, accès IAM (IRSA / Pod Identity)
- Choisir ECS vs EKS vs Lambda

1. Choisir son orchestrateur

Critère	ECS	EKS	Lambda
Complexité	Faible	Élevée (Kubernetes)	Très faible
Portabilité	AWS uniquement	Kubernetes standard, multi-cloud	AWS
Écosystème	AWS natif	Helm, operators, CNCF	Événementiel
Cas typique	Services web/API, workers	Plateforme multi-équipes, K8s existant	Événements courts
Coût de contrôle	0	0,10 $/h/cluster	à l'usage

Modes de calcul : **Fargate** (serverless, pas de nœuds à gérer) ou **EC2** (contrôle, GPU, Spot, densité).

2. ECR

Registre privé OCI : scan de vulnérabilités (basic / enhanced Inspector), **immutabilité des tags**, politiques de cycle de vie, réplication inter-région/compte, chiffrement KMS.

```
REGION=eu-west-3; ACCOUNT=$(aws sts get-caller-identity --query Account --output text)
REG=$ACCOUNT.dkr.ecr.$REGION.amazonaws.com
aws ecr create-repository --repository-name demo/web --image-scanning-configuration
scanOnPush=true \
  --image-tag-mutability IMMUTABLE
aws ecr get-login-password | docker login --username AWS --password-stdin $REG
docker build -t $REG/demo/web:1.0.0 . && docker push $REG/demo/web:1.0.0
```

Dans les pipelines : tag = SHA du commit (traçabilité), jamais `latest` en production.

3. ECS — concepts

```
Cluster
 └── Service (désiré = N, ALB, autoscaling, déploiement)
      └── Task (instance d'une Task Definition)
           └── Containers (image, CPU/mémoire, ports, env, secrets, logs)
```

Élément	Rôle
Task Definition	Plan versionné : image(s), CPU/RAM, `awsvpc`, rôles
Task Execution Role	Utilisé par ECS pour tirer l'image ECR, écrire les logs, lire les secrets
Task Role	Permissions de **votre application** (S3, DynamoDB...)
Service	Maintient N tâches, enregistre dans le target group, déploie
Capacity providers	Fargate / Fargate Spot / ASG EC2

Mode réseau `awsvpc` : chaque tâche a sa propre ENI, son SG et une IP du subnet.

Déploiements

- **Rolling update** (défaut) : `minimumHealthyPercent` / `maximumPercent`, avec **circuit breaker** + rollback automatique.
- **Blue/Green via CodeDeploy** : bascule de trafic ALB (canary, linear), tests avant bascule.
- Secrets : référence à Secrets Manager / Parameter Store dans la task definition (jamais en clair).
- **ECS Exec** : shell dans un conteneur en prod (via SSM), audité.
- **Service Connect** / Cloud Map : découverte de services.

4. EKS

Plan de contrôle Kubernetes managé (multi-AZ). Options de calcul : **managed node groups**, **Fargate profiles**, **Karpenter** (provisioning rapide des nœuds, optimisation de coût), nœuds auto-gérés.

Éléments propres à AWS :

- **VPC CNI** : les pods reçoivent une IP du VPC.
- **AWS Load Balancer Controller** : `Ingress` → ALB, `Service LoadBalancer` → NLB.
- **EKS Pod Identity / IRSA** : un pod endosse un rôle IAM (plus de clés dans les pods).
- **Accès cluster** : *access entries* (API) / `aws-auth` ConfigMap (hérité) ; `aws eks update-kubeconfig`.
- **Add-ons** managés : CoreDNS, kube-proxy, VPC CNI, EBS CSI, observabilité.
- Mises à jour de version : plan de contrôle puis nœuds, tous les ~14 mois par version (support étendu payant).

```
aws eks update-kubeconfig --name demo --region eu-west-3
kubectl get nodes
kubectl create deployment web --image=public.ecr.aws/nginx/nginx:stable --replicas=3
kubectl expose deployment web --port 80 --type LoadBalancer
```

> Un cours Kubernetes complet est disponible dans le dépôt `K8s` ; ici on se concentre sur l'intégration AWS.

5. Sécurité des conteneurs

1. Image minimale, utilisateur non-root, scan à chaque push (ECR + Trivy en CI).
2. Un rôle IAM **par service** (task role / Pod Identity).
3. Réseau : tâches en subnets **privés**, SG limités, endpoints ECR/S3/Logs pour éviter la NAT.
4. Secrets hors image ; fichiers système en lecture seule.
5. Journalisation vers CloudWatch Logs (`awslogs`) ou FireLens.

6. Démos

Démo 01 — Build & push ECR (voir commandes ci-dessus + scan).
Démo 02 — ECS Fargate en 5 minutes via la console (« Create service ») pour voir tous les objets, puis recréation en Terraform.
Démo 03 — Rollback automatique : déployer une image qui échoue au health check ; constater le circuit breaker.

7. Labo 09 — Service ECS Fargate derrière un ALB

Énoncé : déployer une petite API (image publique `nginx` puis votre image ECR) sur ECS Fargate, 2 tâches sur 2 AZ, ALB, logs CloudWatch, autoscaling CPU, circuit breaker.

Code : `labs/lab-ecs/` — application d'exemple dans `labs/lab-ecs/app/`.

```
cd labs/lab-ecs
terraform init && terraform apply
curl $(terraform output -raw alb_url)/
```

Étapes :

1. Vérifier les tâches (`aws ecs list-tasks`) et les logs (`aws logs tail /ecs/lab09 --follow`).
2. Construire l'image de `app/`, la pousser dans ECR (output `ecr_repo`), puis relancer `terraform apply -var image=<URI:tag> -var container_port=8080 -var health_path=/health`. (Le dépôt ECR est créé par le premier `apply` ; l'image est en `linux/amd64` : `docker build --platform linux/amd64`.)
3. Casser volontairement l'image (port erroné) → observer l'échec et le rollback.
4. Ouvrir un shell : `aws ecs execute-command ...`.

5. **Défi** : passer à **Fargate Spot** pour 50 % des tâches (capacity provider strategy).
6. `terraform destroy` 💰.

Validation : 0 downtime lors d'un déploiement (boucle `curl` pendant le déploiement), rollback automatique constaté.

8. Labo 09 bis (optionnel) — EKS avec eksctl

```
eksctl create cluster --name demo --region eu-west-3 --nodes 2 --node-type t3.medium --managed
# ... déployer, exposer via Ingress
eksctl delete cluster --name demo       # 💰 ne pas oublier
```

Erreurs fréquentes

Problème	Cause
Tâche bloquée en `PENDING` / `CannotPullContainerError`	Pas de route vers ECR (NAT/endpoints) ou execution role incomplet
Cibles ALB `unhealthy`	Port conteneur ≠ port target group, SG, chemin de health check
`AccessDenied` dans l'appli	Permissions mises sur l'execution role au lieu du task role
Pods `Pending` sur EKS	Plus d'IP dans le subnet, ressources insuffisantes
Facture NAT forte	Pull d'images ECR via NAT → endpoints ECR + S3

Quiz

1. Différence entre task role et task execution role ?
2. Quand préférer EKS à ECS ?
3. Quel mécanisme évite de laisser un mauvais déploiement actif ?
4. Comment donner à un pod EKS l'accès à S3 sans clé ?
5. Pourquoi ne pas utiliser le tag `latest` ?

Réponses

1. Execution role : ECS (pull ECR, logs, secrets). Task role : le code de l'application.
2. Compétences/outillage Kubernetes existants, portabilité, écosystème (Helm, operators) ; sinon ECS est plus simple.
3. Deployment circuit breaker avec rollback (ou blue/green CodeDeploy).
4. IRSA ou EKS Pod Identity.
5. Non reproductible, non traçable, impossible de revenir en arrière de façon déterministe.

Module 10 — Infrastructure as Code : CloudFormation, CDK, Terraform

Durée : 2 jours · **Niveau** : intermédiaire → avancé · **Coût** : ≈ 0 €
Certifications : DOP (cœur), DVA, SOA, SAA

Objectifs

- Comprendre pourquoi et comment décrire l'infrastructure en code
- Maîtriser **CloudFormation** (stacks, change sets, drift, StackSets)
- Découvrir **AWS CDK** et comparer avec **Terraform**
- Structurer un projet **Terraform** : modules, backend S3 + verrouillage, environnements, tests, scan sécurité

1. Pourquoi l'IaC ?

Reproductibilité · revue de code (Pull Requests) · traçabilité (Git) · détection de dérive · environnements identiques · reprise après sinistre · automatisation CI/CD.

Principes : **déclaratif**, **idempotent**, **immuable**, **versionné**, **testé**, **modulaire**.

2. CloudFormation

Service natif AWS : un **template** (YAML/JSON) → une **stack**.

```yaml
AWSTemplateFormatVersion: "2010-09-09"
Description: Bucket chiffré
Parameters:
  Env: { Type: String, AllowedValues: [dev, prod], Default: dev }
Conditions:
  IsProd: !Equals [!Ref Env, prod]
Resources:
  Bucket:
    Type: AWS::S3::Bucket
    DeletionPolicy: Retain
    Properties:
      VersioningConfiguration:
        Status: !If [IsProd, Enabled, Suspended]
      BucketEncryption:
        ServerSideEncryptionConfiguration:
          - ServerSideEncryptionByDefault: { SSEAlgorithm: AES256 }
      PublicAccessBlockConfiguration:
        BlockPublicAcls: true
        BlockPublicPolicy: true
        IgnorePublicAcls: true
        RestrictPublicBuckets: true
Outputs:
  BucketName:
    Value: !Ref Bucket
    Export: { Name: !Sub "${AWS::StackName}-bucket" }
```

Concepts à retenir

Concept	Rôle
Fonctions intrinsèques	`!Ref`, `!GetAtt`, `!Sub`, `!Join`, `!If`, `!FindInMap`, `!ImportValue`
Change set	Prévisualiser les modifications avant exécution (≈ `terraform plan`)
Drift detection	Détecte les modifications manuelles hors stack
DeletionPolicy / UpdateReplacePolicy	`Retain`, `Snapshot` pour protéger les données
Stack policy, **termination protection**	Empêcher des mises à jour/suppressions accidentelles
Rollback	Retour automatique en cas d'échec de création/mise à jour
Nested stacks, **modules**, **StackSets**	Réutilisation, déploiement multi-comptes/régions
Custom resources	Lambda pour ressources non supportées
cfn-lint, **cfn-guard**	Lint / politiques « policy as code »

```bash
aws cloudformation deploy --template-file bucket.yaml --stack-name demo-bucket \
  --parameter-overrides Env=dev --capabilities CAPABILITY_NAMED_IAM
aws cloudformation describe-stack-events --stack-name demo-bucket
aws cloudformation detect-stack-drift --stack-name demo-bucket
```

3. AWS CDK

Définir l'infrastructure dans un **vrai langage** (TypeScript, Python, Java, C#, Go) → synthétisée en CloudFormation (`cdk synth`). *Constructs* L1 (CFN brut), **L2** (abstractions avec bonnes pratiques par défaut), L3 (patterns).

```
# app.py (CDK Python)
from aws_cdk import App, Stack, aws_s3 as s3, RemovalPolicy
from constructs import Construct

class DataStack(Stack):
    def __init__(self, scope: Construct, id: str, **kw):
        super().__init__(scope, id, **kw)
        s3.Bucket(self, "Data", versioned=True, encryption=s3.BucketEncryption.S3_MANAGED,
                  block_public_access=s3.BlockPublicAccess.BLOCK_ALL,
                  removal_policy=RemovalPolicy.RETAIN)
app = App(); DataStack(app, "DataStack"); app.synth()
```

```
npm i -g aws-cdk && cdk bootstrap && cdk diff && cdk deploy && cdk destroy
```

4. Terraform

Outil **multi-cloud** de HashiCorp (licence BSL ; fork communautaire **OpenTofu**). Langage HCL, **state** décrivant ce qui est géré, ~3000 providers.

Cycle : `init` → `fmt` / `validate` → `plan` → `apply` → `destroy`.

Concepts

Notion	Détail
Provider	Plugin d'API (`hashicorp/aws`), version épinglée
Resource / Data source	Créer / lire
Variables, locals, outputs	Entrées, calculs, sorties
State	Fichier de correspondance code ↔ réel. **Sensible**, jamais dans Git
Backend distant	S3 + verrouillage (DynamoDB ou verrou natif S3 `use_lockfile` dans les versions récentes)
Modules	Composants réutilisables et versionnés
`count` / `for_each`	Répétition ; préférer `for_each` (clés stables)
`lifecycle`	`prevent_destroy`, `create_before_destroy`, `ignore_changes`
`moved`, `import`, `removed`	Refactoring et adoption de l'existant
Workspaces	Séparation légère ; préférer des dossiers/états distincts par environnement

Backend S3 (à amorcer une fois)

```
terraform {
  backend "s3" {
    bucket         = "acme-tfstate-<account>"
    key            = "prod/network/terraform.tfstate"
    region         = "eu-west-3"
    encrypt        = true
    dynamodb_table = "tf-locks"   # ou: use_lockfile = true (Terraform ≥ 1.10)
  }
}
```

Bucket : versioning, chiffrement KMS, accès restreint, aucune suppression publique.

Structure recommandée

```
infra/
├── modules/            # briques réutilisables (vpc, ecs-service, rds…)
│   └── vpc/{main,variables,outputs}.tf
├── envs/
│   ├── dev/{main.tf,backend.tf,terraform.tfvars}
│   └── prod/…
└── .github/workflows/terraform.yml   # plan sur PR, apply sur merge
```

Qualité et sécurité

`terraform fmt -check` · `validate` · **tflint** · **checkov / tfsec / trivy config** · tests natifs (`terraform test`) ou **Terratest** · estimation de coûts (**Infracost**) · politiques (OPA/Sentinel).

5. Comparatif

	CloudFormation	CDK	Terraform
Langage	YAML/JSON	TS/Python/Java/…	HCL
Multi-cloud	Non	Non (CDKTF existe)	**Oui**
État	Géré par AWS	Géré par AWS (via CFN)	**À gérer** (backend)
Nouveautés AWS	Jour 1	Via CFN	Délai possible
Atouts	Intégration native, StackSets, rollback	Productivité, abstraction	Écosystème, lisibilité, modules

Examen AWS : **CloudFormation/CDK/SAM** sont attendus. Terraform est le standard du marché : maîtrisez les deux.

6. Démos

Démo 01 — Stack CloudFormation : `demos/bucket.yaml` — déploiement, change set, drift (modifier le bucket à la main), suppression avec `Retain`.
Démo 02 — Terraform : le state : `terraform state list`, `terraform import`, `moved`, et ce qui se passe quand on supprime le state.
Démo 03 — Backend distant : migrer un state local vers S3 (`terraform init -migrate-state`).

7. Labo 10 — Un module Terraform réutilisable et un backend distant

Énoncé :

1. Amorcer le backend (`labs/lab-terraform/bootstrap`) : bucket d'état versionné/chiffré + verrouillage.
2. Utiliser le **module** `modules/s3-bucket-secure` pour créer deux buckets (`dev`, `prod`) avec `for_each`, depuis `labs/lab-terraform/envs/demo`.
3. Lancer `terraform plan` avec `-out`, lire le plan, appliquer.
4. Modifier un bucket à la main (console) puis `terraform plan` → constater la **dérive**.
5. Refactorer avec un bloc `moved` sans destruction.
6. Passer `tflint` / `checkov -d .` et corriger les alertes.
7. **Défi** : écrire un test `terraform test` qui vérifie que le chiffrement est actif.
8. Détruire (démo d'abord, puis bootstrap).

Code : `labs/lab-terraform/`

Validation : `plan` = « No changes » après apply ; dérive détectée et corrigée.

Erreurs fréquentes

Problème	Remède
State perdu / divergent	Backend distant versionné + verrou ; `terraform import`
Secrets dans le state	Chiffrer le backend, restreindre l'accès, utiliser `manage_master_user_password`, Secrets Manager
`count` qui recrée tout en retirant un élément	Utiliser `for_each`
Drift permanent	Interdire les changements manuels (SCP/rôles lecture seule), détecter via `plan` planifié
Stack CloudFormation en `ROLLBACK_COMPLETE`	Supprimer puis recréer ; lire les events
Un seul gros state	Découper par domaine (réseau / données / applis)

Quiz

1. Quelle commande prévisualise les changements avec CloudFormation ?
2. À quoi sert le verrouillage du state ?
3. Pourquoi `for_each` plutôt que `count` ?
4. Que fait `cdk synth` ?
5. Comment adopter une ressource existante dans Terraform ?

Réponses

1. Un *change set* (`aws cloudformation deploy --no-execute-changeset` ou `create-change-set`).
2. Éviter deux `apply` concurrents qui corrompent le state.
3. Les éléments sont identifiés par une clé stable, pas par un index qui bouge.
4. Génère le template CloudFormation depuis le code.
5. `terraform import` ou un bloc `import {}` puis écrire la ressource dans le code.

Module 11 — CI/CD sur AWS

Durée : 1,5 jour · **Niveau** : intermédiaire · **Coût** : ≈ 0 € (CodeBuild/CodePipeline : quota gratuit limité)
Certifications : **DOP-C02 (cœur)**, DVA, SOA

Objectifs

- Concevoir un pipeline : *build → test → scan → déploiement → vérification → rollback*
- Utiliser **GitHub Actions avec OIDC** pour déployer sur AWS **sans clé**
- Connaître la suite **CodeBuild / CodePipeline / CodeDeploy / CodeArtifact**
- Appliquer les stratégies de déploiement : **rolling, blue/green, canary**, et les **feature flags**

1. Le pipeline idéal

```
Commit → [Lint + tests unitaires] → [Build image/artefact] → [Scan SAST/SCA/IaC/image]
       → [Publication artefact signé (ECR/S3/CodeArtifact)]
       → [Déploiement dev auto] → [Tests d'intégration/smoke]
       → [Approbation] → [Déploiement prod progressif] → [Vérif. métriques → rollback auto]
```

Principes DevOps : **build once, deploy many** (même artefact dans tous les environnements), pipelines **as code**, secrets hors code, environnements isolés (comptes séparés), traçabilité (commit SHA ↔ version déployée).

2. Services AWS

Service	Rôle
CodeCommit	Dépôt Git (⚠️ fermé aux nouveaux clients en 2024 puis rouvert ; vérifiez sa disponibilité, GitHub/GitLab restent le choix courant)
CodeBuild	Build/tests managés, conteneurs éphémères, `buildspec.yml`
CodeDeploy	Déploiement vers EC2/on-prem, Lambda, ECS (rolling, blue/green, canary) via `appspec`
CodePipeline	Orchestration (source → build → test → deploy → approbation), V2 déclencheurs/variables
CodeArtifact	Dépôt de paquets (npm, PyPI, Maven…)
ECR / S3	Stockage d'artefacts
EC2 Image Builder	Pipelines d'AMI/conteneurs
CloudFormation / CDK Pipelines	Déploiement d'infrastructure
Elastic Beanstalk / App Runner	PaaS avec déploiement intégré

buildspec.yml (CodeBuild)

```
version: 0.2
env:
  variables: { APP: lab11 }
phases:
  install:
    runtime-versions: { python: 3.12 }
  pre_build:
    commands:
      - pip install -r requirements.txt
      - pytest -q
      - aws ecr get-login-password | docker login --username AWS --password-stdin
$ECR_REGISTRY
  build:
    commands:
      - docker build -t $ECR_REGISTRY/$APP:$CODEBUILD_RESOLVED_SOURCE_VERSION .
  post_build:
    commands:
      - docker push $ECR_REGISTRY/$APP:$CODEBUILD_RESOLVED_SOURCE_VERSION
artifacts:
  files: [ appspec.yaml, taskdef.json ]
reports:
  junit: { files: ["report.xml"], file-format: JUNITXML }
cache:
  paths: [ "/root/.cache/pip/**/*" ]
```

3. Stratégies de déploiement

Stratégie	Principe	Risque / coût
All-at-once	Tout d'un coup	Indisponibilité possible, retour lent
Rolling	Par lots	Versions mixtes temporaires
Rolling with extra batch / immutable	Nouvelles instances puis bascule	Capacité temporaire supplémentaire
Blue/Green	Deux environnements, bascule du trafic	Coût doublé brièvement, retour instantané
Canary	5-10 % du trafic puis le reste	Nécessite métriques/alarmes
Linear	Augmentation par paliers (10 %/min)	
Feature flags	Découpler déploiement et activation	AppConfig

CodeDeploy Lambda : alias avec `Canary10Percent5Minutes`, alarmes CloudWatch → **rollback automatique**.

4. GitHub Actions + OIDC → AWS (sans clé longue durée)

Principe : GitHub émet un **JWT** pour chaque job ; AWS IAM l'accepte via un *OIDC provider* et un rôle dont la trust policy filtre par dépôt/branche/environnement.

1. Créer le provider `token.actions.githubusercontent.com` (audience `sts.amazonaws.com`).
2. Créer un rôle avec une trust policy limitée à `repo:ORG/REPO:ref:refs/heads/main`.
3. Dans le workflow : `permissions: id-token: write` + `aws-actions/configure-aws-credentials`.

Code d'infrastructure : `labs/lab-oidc/` · Workflow : `labs/lab-oidc/workflow-example.yml`

> Sécurité : ne jamais utiliser un `sub` en joker large (`repo:ORG/*`) sur un rôle puissant ; séparer les rôles *plan* (lecture seule, PR) et *apply* (écriture, branche `main` + environnement protégé avec approbation).

5. Pipeline Terraform sur PR (patron courant)

```
PR ouverte   → fmt, validate, tflint, checkov, plan (rôle lecture) → commentaire du plan
Merge main   → apply (rôle écriture, environnement avec approbation manuelle)
Planifié     → plan de détection de dérive + alerte
```

Un exemple complet est fourni dans `labs/lab-oidc/terraform-pipeline.yml`.

6. Sécurité de la chaîne d'approvisionnement

- Épingler les actions GitHub par SHA, Dependabot/Renovate.
- Scan : SAST (CodeQL, Semgrep), SCA (Dependabot, Trivy), secrets (gitleaks), images (ECR/Inspector, Trivy), IaC (checkov).
- **SBOM** et **signature** d'images (Sigstore cosign / AWS Signer).
- Principe du moindre privilège pour les rôles de pipeline ; comptes de déploiement séparés ; approbations pour la prod.

7. Démos

Démo 01 — CodeBuild en 10 minutes : projet relié à un dépôt public, buildspec simple, lecture des logs et rapports de tests.
Démo 02 — Blue/Green ECS avec CodeDeploy : bascule 10 % puis 100 %, test d'un rollback (alarme).
Démo 03 — OIDC en action : `aws sts get-caller-identity` dans un workflow, observer le `sub` du token.

8. Labo 11 — Déployer depuis GitHub Actions sans clé

Énoncé :

1. Appliquer `labs/lab-oidc` (variables : `github_org`, `github_repo`) → crée provider OIDC + rôles *plan*/*apply* + bucket de déploiement.
2. Copier `workflow-example.yml` dans `.github/workflows/` d'un dépôt de test, renseigner l'ARN du rôle.
3. Le workflow : tests → assume le rôle → `aws s3 sync` d'un site statique (ajoutez ensuite une invalidation CloudFront si vous avez une distribution).
4. Prouver que la branche `feature/x` **ne peut pas** assumer le rôle apply (lire l'erreur `Not authorized to perform sts:AssumeRoleWithWebIdentity`).
5. **Défi** : ajouter un environnement GitHub « production » avec reviewer obligatoire et le conditionner dans la trust policy (`sub = repo:ORG/REPO:environment:production`).

Validation : aucun secret AWS dans les paramètres du dépôt ; déploiement réussi depuis `main`, refusé depuis une autre branche.

Erreurs fréquentes

Problème	Cause
`Could not assume role with OIDC`	`sub` / `aud` ne correspondent pas, `id-token: write` oublié
Pipeline qui déploie un artefact différent par environnement	Rebuild à chaque étape (violer « build once »)
Déploiement bloqué sur ECS	Health check, quotas, taskdef invalide
Secrets dans les logs	Masquer, utiliser Secrets Manager, `set +x`
Rollback impossible	Artefacts supprimés / migrations de base non rétro-compatibles → migrations *expand/contract*

Quiz

1. Pourquoi OIDC plutôt que des clés d'accès dans GitHub Secrets ?
2. Différence blue/green et canary ?
3. Quel fichier décrit les étapes CodeBuild ?
4. Comment CodeDeploy sait-il qu'il doit revenir en arrière ?
5. Qu'est-ce que « build once, deploy many » ?

Réponses

1. Credentials temporaires, aucun secret à stocker/faire tourner, restriction par repo/branche.
2. Blue/green : bascule complète entre deux environnements ; canary : fraction progressive du trafic.
3. `buildspec.yml`.
4. Alarmes CloudWatch associées au déploiement ou échec des hooks de validation.
5. On construit un artefact unique, promu à l'identique dev → staging → prod.

Module 12 — Observabilité, audit et conformité

Durée : 1 jour · **Niveau** : intermédiaire · **Coût** : ≈ 0–2 € (logs et métriques personnalisées facturés)
Certifications : **SOA (cœur)**, DOP, SAA, SCS

Objectifs

- Mettre en place **métriques, logs, traces, alarmes, tableaux de bord**
- Distinguer **CloudWatch** (performance), **CloudTrail** (qui a fait quoi), **Config** (état et conformité)
- Construire une chaîne **alerte → notification → remédiation automatique**
- Définir des **SLI/SLO** et réduire le bruit

1. Les trois piliers + audit

Besoin	Service
Métriques (séries temporelles)	CloudWatch Metrics
Logs	CloudWatch Logs (+ Logs Insights)
Traces distribuées	X-Ray / OpenTelemetry (ADOT)
Qui a appelé quelle API ?	**CloudTrail**
Quelle est la config actuelle/passée, conforme ?	**AWS Config**
Santé de la plateforme AWS	Health Dashboard / EventBridge
Recommandations	Trusted Advisor, Compute Optimizer
Cartographie d'applications	Application Signals, CloudWatch ServiceLens

2. CloudWatch Metrics

- Espace de noms (`AWS/EC2`), **dimensions** (InstanceId), statistiques (Average, p99), période (60 s standard, 1 s haute résolution).
- EC2 : métriques de base toutes les 5 min (1 min en *detailed*). **Mémoire et disque** : nécessite l'**agent CloudWatch**.
- **Métriques personnalisées** (`put-metric-data`) et **EMF** (Embedded Metric Format : métriques via des logs JSON).
- **Metric math**, **anomaly detection**.

3. Alarmes

États `OK` / `ALARM` / `INSUFFICIENT_DATA` ; période × points de données (ex. 3 sur 5). Actions : SNS, Auto Scaling, EC2 (reboot/recover), Systems Manager OpsItem. **Alarmes composites** pour réduire le bruit.

```
aws cloudwatch put-metric-alarm --alarm-name alb-5xx --namespace AWS/ApplicationELB \
  --metric-name HTTPCode_Target_5XX_Count --dimensions Name=LoadBalancer,Value=app/xxx/yyy \
  --statistic Sum --period 60 --evaluation-periods 3 --datapoints-to-alarm 2 \
  --threshold 10 --comparison-operator GreaterThanThreshold --treat-missing-data notBreaching
\
  --alarm-actions arn:aws:sns:eu-west-3:123456789012:alerts
```

4. CloudWatch Logs

Organisation : **log group** (rétention, KMS) → **log streams**. Sources : agent CW, Lambda, ECS `awslogs`, VPC Flow Logs, Route 53…

- **Rétention** : définissez-la toujours (défaut = illimité = coûteux).
- **Metric filters** : transformer un motif de log en métrique (ex. `ERROR`).
- **Subscription filters** : flux temps réel vers Lambda/Kinesis/Firehose/OpenSearch.
- **Logs Insights** : requêtes.

```
fields @timestamp, @message
| filter @message like /ERROR/
| stats count() as erreurs by bin(5m)
| sort @timestamp desc
```

- Export vers S3 pour archivage ; classe de logs *Infrequent Access*.

5. Traçage

X-Ray : trace de bout en bout (API GW → Lambda → DynamoDB…), service map, analyse de latence. Instrumentation via SDK X-Ray ou **OpenTelemetry (ADOT collector)**.

6. CloudTrail

Journal des appels d'API (console, CLI, SDK) : **qui, quoi, quand, d'où**.

- **Événements de gestion** : activés par défaut (90 jours dans *Event history*).
- **Trail** vers S3 (multi-région, toutes les régions, **validation d'intégrité**) pour conservation longue, + CloudWatch Logs pour alerter.
- **Événements de données** (S3 objets, Lambda invoke…) : opt-in, volumineux.
- **CloudTrail Lake** : requêtes SQL sur les événements.
- **Organization trail** : centralisation multi-comptes.

Cas d'usage : « Qui a supprimé ce groupe de sécurité ? » → `lookup-events` ou Athena.

7. AWS Config

Enregistre l'**historique de configuration** des ressources et évalue des **règles** de conformité (managées ou Lambda/Guard) : S3 non chiffré, SG ouvert sur 22, volumes EBS non chiffrés… **Remédiation automatique** via Systems Manager Automation. **Conformance packs**, agrégateur multi-comptes.

> Retenir : **CloudTrail = actions (API) · Config = état/conformité · CloudWatch = performance/opérationnel**.

8. Automatisation événementielle

```
EC2 passe en 'stopped' → EventBridge rule → SNS (e-mail) + Lambda/SSM Automation (remédiation)
Config règle non conforme → EventBridge → SSM Automation (corriger)
GuardDuty finding HIGH → EventBridge → Lambda (isoler l'instance) + SNS
```

9. SLI / SLO et alertes utiles

- **SLI** : proportion de requêtes réussies, latence p99, saturation.
- **SLO** : objectif (ex. 99,9 %/30 j) → **budget d'erreur** → guide les décisions de déploiement.
- Alerter sur les **symptômes** (erreurs utilisateurs, latence) plutôt que sur chaque cause ; alertes **actionnables** avec runbook.
- Les « 4 signaux dorés » : latence, trafic, erreurs, saturation.

10. Démos

Démo 01 — Alarme + SNS : CPU d'une instance → e-mail ; provoquer avec `stress`.
Démo 02 — Logs Insights : requête sur les logs Lambda du module 08 (erreurs par minute, p95 de durée).
Démo 03 — Enquête CloudTrail : retrouver l'auteur d'une suppression (`demos/demo-01-cloudtrail.sh`).
Démo 04 — Config rule + remédiation : règle `s3-bucket-public-read-prohibited`.

11. Labo 12 — Supervision et remédiation automatique

Énoncé : instrumenter l'application du module 07/08 et automatiser les réactions.

Prérequis : la Lambda `lab08-api` du module 08 doit être déployée **et invoquée au moins une fois** (le log group doit exister pour le metric filter) ; sinon passez `-var lambda_name=<autre fonction>`.

Code : `labs/lab-observability/` crée : topic SNS + abonnement e-mail, dashboard, alarme sur erreurs Lambda, metric filter sur logs, trail CloudTrail → S3, règle EventBridge qui notifie à chaque `StopInstances`.

Étapes :

1. `terraform apply -var email=vous@exemple.com`, confirmer l'abonnement SNS.
2. Provoquer des erreurs Lambda (`aws lambda invoke` avec payload qui plante) → recevoir l'alerte.
3. Arrêter une instance → recevoir la notification EventBridge.
4. Requête Logs Insights : top 5 des erreurs.
5. CloudTrail : retrouver votre propre `StopInstances` (qui, IP, userAgent).
6. **Défi** : alarme composite « (5xx ALB élevées) ET (latence p99 élevée) ».
7. `terraform destroy`.

Validation : vous recevez 2 notifications distinctes et retrouvez l'événement correspondant dans CloudTrail.

Erreurs fréquentes

Problème	Cause
Pas de métrique mémoire	Agent CloudWatch non installé/configuré
Alarme en `INSUFFICIENT_DATA`	Pas de données ; `treat-missing-data` à ajuster
Facture CloudWatch élevée	Logs verbeux sans rétention, métriques à forte cardinalité
Pas d'historique > 90 j	Aucun trail vers S3
Alertes ignorées	Trop de bruit → SLO, alarmes composites, seuils pertinents

Quiz

1. Quel service répond à « qui a supprimé ce bucket ? »
2. Comment obtenir la mémoire d'une instance EC2 dans CloudWatch ?
3. Quelle différence entre metric filter et subscription filter ?
4. Que fait AWS Config que CloudTrail ne fait pas ?
5. Qu'est-ce qu'un budget d'erreur ?

Réponses

1. CloudTrail.
2. Installer l'agent CloudWatch (métriques personnalisées).
3. Le metric filter crée une métrique à partir des logs ; le subscription filter diffuse les logs en temps réel vers une destination.
4. Historique de configuration et évaluation de conformité (avec remédiation).
5. Le taux d'indisponibilité toléré par le SLO ; consommé, on ralentit les déploiements à risque.

Module 13 — Sécurité avancée

Durée : 1,5 jour · **Niveau** : avancé · **Coût** : ≈ 1–3 € (KMS 1 $/clé/mois, GuardDuty/Config après essai)
Certifications : SCS (cœur), DOP, SAA, SOA

Objectifs

- Maîtriser le **chiffrement** (KMS, envelope encryption, TLS/ACM)
- Gérer les **secrets** (Secrets Manager, Parameter Store)
- Protéger le périmètre (**WAF, Shield**) et détecter les menaces (**GuardDuty, Inspector, Macie, Security Hub**)
- Appliquer **Security by design** : défense en profondeur, automatisation de la conformité, réponse à incident

1. Cadre : défense en profondeur

```
Identité (IAM, MFA, SSO)  →  Réseau (VPC, SG, WAF)  →  Hôte/Conteneur (patch, EDR, Inspector)
  →  Application (code, dépendances)  →  Données (KMS, classification)  →  Détection & réponse
```

Principes : moindre privilège, zero trust, chiffrement partout, **tout en code**, journaliser et alerter, automatiser la remédiation.

2. AWS KMS

Service de gestion de clés (HSM validés FIPS 140-3). Les clés **ne sortent jamais** de KMS non chiffrées.

Type de clé	Gestion	Rotation	Coût
AWS owned	AWS (invisible)	AWS	gratuit
AWS managed (`aws/s3`)	AWS dans votre compte	auto (annuelle)	gratuit
Customer managed (CMK)	Vous (policy, alias, grants)	optionnelle (auto annuelle)	1 $/mois + requêtes
Import / External key store	Vous	manuelle	

Envelope encryption

```
GenerateDataKey ➞ (clé de données en clair + chiffrée par la CMK)
  chiffrer les données avec la clé en clair (AES-256 en local), la jeter de la mémoire
  stocker : données chiffrées + clé de données CHIFFRÉE
Déchiffrer : KMS Decrypt(clé chiffrée) → clé en clair → déchiffrer
```

Raison : KMS chiffre directement ≤ 4 Ko ; au-delà on utilise des clés de données.

Policies de clé

Une CMK a **sa propre policy de ressource** (obligatoire) : sans elle, même les admins IAM ne peuvent rien. Séparer **administrateurs de clé** et **utilisateurs de clé** ; autoriser les services via `kms:ViaService`. **Contexte de chiffrement** (EncryptionContext) pour lier un chiffrement à un usage. Les clés sont **régionales** ; multi-region keys pour la réplication.

3. Secrets

	Secrets Manager	SSM Parameter Store
Rotation automatique	**Oui** (RDS, Redshift, Lambda perso)	Non natif
Coût	~0,40 $/secret/mois	Standard gratuit
Taille	64 Ko	4 Ko (8 Ko avancé)
Multi-région	Réplication native	Non
Usage	Mots de passe BD, clés API	Configuration, flags, petits secrets (`SecureString`)

Bonnes pratiques : ne **jamais** mettre de secret dans le code, l'image, la variable d'environnement en clair, les logs ou Git ; lire à l'exécution via le rôle IAM ; mettre en cache ; rotation.

4. Certificats et TLS

ACM : certificats publics **gratuits**, renouvellement automatique, validation DNS (Route 53), utilisables avec ALB, CloudFront (us-east-1), API Gateway. Les clés privées ne sont pas exportables (sauf ACM Private CA/exportable payant). TLS 1.2+ partout, politiques de sécurité ELB modernes, HSTS.

5. Protection du périmètre

Service	Rôle
AWS WAF	Pare-feu applicatif L7 (ALB, CloudFront, API GW, AppSync) : règles managées (OWASP, bot control), limitation de débit, géo-blocage
Shield Standard / Advanced	DDoS L3/L4 (gratuit) / protection avancée, équipe SRT, remboursement de coûts
Network Firewall	Filtrage réseau stateful dans le VPC, IPS
Firewall Manager	Gestion centralisée WAF/SG/Network Firewall
Security Groups / NACL	Voir module 03
VPC endpoints + policies	Limiter l'accès aux services AWS aux seules ressources attendues

6. Détection et conformité

Service	Détecte / apporte
GuardDuty	Menaces (comptes compromis, crypto-mining, exfiltration) via CloudTrail, VPC Flow, DNS, EKS, S3, Lambda…
Inspector	Vulnérabilités (EC2, ECR, Lambda)
Macie	Données sensibles dans S3 (PII)
Security Hub	Agrégation et score de conformité (CIS, AWS FSBP, PCI) ; findings au format ASFF
Detective	Investigation (graphes de comportement)
IAM Access Analyzer	Accès externes, droits inutilisés, validation de policies
Config	Conformité de configuration (module 12)
Audit Manager / Artifact	Preuves de conformité / rapports AWS (SOC, ISO)

7. Réponse à incident (patron)

1. **Détecter** : GuardDuty/Security Hub → EventBridge.
2. **Contenir** : isoler l'instance (SG de quarantaine), révoquer les sessions du rôle (policy `AWSRevokeOlderSessions`), désactiver les clés.
3. **Préserver** : snapshot EBS, copie des logs, capture mémoire si nécessaire.
4. **Éradiquer / récupérer** : reconstruire depuis l'IaC et des images saines, faire tourner les secrets.
5. **Apprendre** : post-mortem sans blâme, règles de détection supplémentaires.

Compte compromis : root + MFA, révoquer clés, CloudTrail pour mesurer l'étendue, contacter AWS Support.

8. Checklist « compte sain »

- ☐ MFA root, pas de clés root, contacts de sécurité
- ☐ Identity Center, pas d'utilisateurs IAM humains
- ☐ Block Public Access S3 au niveau du **compte**
- ☐ Chiffrement EBS **par défaut**, S3 chiffré, RDS chiffré
- ☐ IMDSv2 obligatoire
- ☐ CloudTrail organisationnel + Config + GuardDuty + Security Hub activés dans toutes les régions
- ☐ Pas de SG ouvert 0.0.0.0/0 sur 22/3389/BD
- ☐ Secrets dans Secrets Manager, rotation active
- ☐ SCP de garde-fous (module 15)

9. Démos

Démo 01 — Envelope encryption à la main : `demos/demo-01-kms-envelope.sh`.
Démo 02 — Secrets Manager : création, lecture dynamique depuis Lambda, rotation.
Démo 03 — GuardDuty : générer des *sample findings* (`aws guardduty create-sample-findings`) et déclencher une alerte EventBridge.
Démo 04 — WAF : bloquer une injection SQL de test (`?id=1' OR '1'='1`) sur un ALB.

10. Labo 13 — Durcir un environnement

Énoncé : appliquer un socle de sécurité avec Terraform.

Code : `labs/lab-security/` crée : CMK avec rotation, dont la policy délègue la gestion à IAM (l'accès est alors donné rôle par rôle), secret chiffré, chiffrement EBS par défaut, Block Public Access au niveau compte, GuardDuty (échoue si un détecteur existe déjà : importez-le), WAF (ACL non associée : à attacher à un ALB/API) avec règles managées + limitation de débit, et règle Config « SSH ouvert » (crée aussi l'enregistreur Config : un seul par région et par compte).

Étapes :

1. `terraform apply`.
2. Tenter de lire le secret avec un rôle sans `kms:Decrypt` → refus malgré `secretsmanager:GetSecretValue`.
3. Créer un SG ouvert sur 22 → voir la règle Config signaler la non-conformité.
4. Générer des sample findings GuardDuty.
5. Scanner le code Terraform du dépôt : `checkov -d .` ; corriger 3 alertes.
6. **Défi** : rédiger un runbook d'incident « clé d'accès exposée sur GitHub » (étapes, commandes, délais).
7. `terraform destroy`.

Validation : vous démontrez les trois couches de blocage (IAM, KMS, réseau).

Erreurs fréquentes

Problème	Cause
`AccessDeniedException` sur secret malgré IAM	La policy de la CMK n'autorise pas le rôle
Données illisibles après suppression d'une clé	Suppression KMS (7-30 j de délai) : désactiver d'abord
Certificat introuvable pour CloudFront	ACM en **us-east-1**
Alertes GuardDuty ignorées	Pas d'intégration EventBridge/Slack/ticket
Clé AWS commitée	Révoquer immédiatement, ne pas se contenter de supprimer le commit

Quiz

1. Pourquoi l'envelope encryption ?
2. Différence Secrets Manager / Parameter Store ?
3. Quel service surveille les comportements anormaux d'un compte ?
4. Qui peut administrer une CMK dont la policy est vide pour l'IAM ?
5. Que contient une réponse à incident en premier : éradiquer ou contenir ?

Réponses

1. KMS ne chiffre que de petits volumes ; on chiffre les données localement avec une clé de données protégée par la CMK, ce qui limite les appels et l'exposition.
2. Secrets Manager : rotation, réplication, payant ; Parameter Store : configuration, gratuit en standard.
3. GuardDuty.
4. Personne via IAM seul (sauf l'activation de la délégation IAM dans la policy de clé).
5. Contenir, puis préserver les preuves, éradiquer, récupérer.

Module 14 — Architecture, Well-Architected, résilience et FinOps

Durée : 1 jour · **Niveau** : avancé · **Coût** : 0 €
Certifications : SAA, SAP, DOP (très présent)

Objectifs

- Appliquer les **6 piliers du Well-Architected Framework**
- Concevoir pour la **haute disponibilité** et la **reprise après sinistre** (RTO/RPO)
- Choisir un **patron d'architecture** adapté
- Mettre en œuvre le **FinOps** : visibilité, optimisation, gouvernance

1. Well-Architected Framework

Pilier	Question centrale	Pratiques clés
Excellence opérationnelle	Comment exploiter et améliorer ?	IaC, CI/CD, runbooks, observabilité, post-mortems
Sécurité	Comment protéger données et systèmes ?	IAM, chiffrement, détection, réponse à incident
Fiabilité	Comment résister aux pannes ?	Multi-AZ, auto-recovery, sauvegardes, tests de panne, quotas
Efficacité des performances	Comment utiliser les bonnes ressources ?	Bon type/taille, cache, CDN, serverless, mesures
Optimisation des coûts	Comment éviter les dépenses inutiles ?	Right-sizing, Savings Plans, Spot, tags, budgets
Durabilité	Comment réduire l'impact ?	Graviton, utilisation élevée, régions sobres, cycle de vie des données

Outil : **AWS Well-Architected Tool** (console) — revue d'une charge de travail, plan d'amélioration ; *lenses* (Serverless, SaaS, Data Analytics…).

2. Résilience et reprise après sinistre

RPO : perte de données maximale tolérée. **RTO** : durée maximale d'indisponibilité tolérée.

Stratégie DR	RPO / RTO	Coût	Principe
Backup & Restore	heures / heures-jours	$	Sauvegardes copiées dans une autre région ; reconstruction via IaC

Stratégie DR	RPO / RTO	Coût	Principe
Pilot Light	minutes / dizaines de min	$$	BD répliquée, le reste « éteint » (AMI, IaC)
Warm Standby	secondes-minutes / minutes	$$$	Version réduite active en permanence, montée en charge au besoin
Multi-site actif-actif	~0 / ~0		Deux régions servent le trafic (Route 53, Global Tables, Aurora Global)

Niveaux de panne à envisager : instance → **AZ** → **région** → compte (corruption, compromission) → fournisseur. Chaque niveau a sa parade : ASG, Multi-AZ, DR inter-région, comptes de sauvegarde isolés (Backup Vault Lock).

Outils : **AWS Backup**, **Elastic Disaster Recovery (DRS)**, S3 CRR, Aurora Global, DynamoDB Global Tables, Route 53 ARC, **Fault Injection Service** (chaos engineering), **Resilience Hub**.

Quotas (Service Quotas)

Un DR qui échoue faute de quotas dans la région cible est classique : demandez-les à l'avance.

3. Patrons d'architecture

Patron	Description	Briques AWS
3-tiers	Web / App / Données	CloudFront, ALB, ASG/ECS, RDS Multi-AZ
Serverless	Sans gestion de serveurs	API GW, Lambda, DynamoDB, SQS
Événementiel	Découplage par événements	EventBridge, SNS/SQS, Lambda
Microservices	Services indépendants	ECS/EKS, ALB, App Mesh/Service Connect
Data lake	Analyse à grande échelle	S3, Glue, Athena, Lake Formation, Redshift
Hub & spoke réseau	Réseau multi-VPC	Transit Gateway, Network Firewall
Multi-comptes	Isolation par charge	Organizations, Control Tower (module 15)
Strangler fig	Migration progressive d'un monolithe	CloudFront/ALB + routage

Principes : **découpler** (files, événements), **sans état**, **automatiser**, **concevoir pour la panne**, **mesurer avant d'optimiser**, **données au plus près du calcul**.

4. Les 7 R de la migration

Retire · Retain · Rehost (lift & shift) · Relocate · Repurchase · **Replatform** · **Refactor**. Outils : Migration Hub, Application Migration Service (MGN), DMS, DataSync, Snow Family.

5. FinOps

Cycle Inform → Optimize → Operate

1. **Visibilité** : tags de coût obligatoires (`env` , `team` , `app` , `owner`) activés comme *cost allocation tags*, **Cost Explorer**, **CUR** (Cost & Usage Report) + Athena, **Budgets** par équipe, **Cost Anomaly Detection**.
2. **Optimisation** :

Levier	Gain
Éteindre / supprimer l'inutile (dev la nuit, volumes et EIP orphelins)	Immédiat
Right-sizing (Compute Optimizer)	20–40 %
Savings Plans / RI sur la charge de base	jusqu'à 70 %
Spot (CI, batch, workers)	jusqu'à 90 %
Graviton	~20 %
Stockage : S3 Intelligent-Tiering, gp3 vs gp2, snapshots anciens	10–50 %
Réseau : endpoints (éviter NAT), CloudFront, 1 AZ pour trafics lourds si possible	variable
Serverless/scale to zero pour charges intermittentes	variable
Rétention des logs, métriques à forte cardinalité	variable

3. **Gouvernance** : SCP limitant les régions et types coûteux, quotas, Budget Actions, revues mensuelles, *unit economics* (coût par client / par transaction).

Pièges de coût fréquents

NAT Gateway (trafic de données) · transferts inter-AZ/inter-régions/Internet · logs sans rétention · snapshots oubliés · load balancers inactifs · clusters EKS oubliés · Multi-AZ en dev · Elastic IP inutilisées · requêtes S3 massives.

6. Démos

Démo 01 — Well-Architected Tool : mener une revue express (sécurité + fiabilité) sur le projet du module 07, générer le plan d'amélioration.
Démo 02 — Cost Explorer : regrouper par service/tag, repérer les 3 plus gros postes, activer Anomaly Detection (`demos/demo-01-cost-cli.sh`).
Démo 03 — Test de panne : arrêter une AZ « simulée » (terminer les instances d'une AZ) et mesurer la reprise ; optionnel avec AWS FIS.

7. Labo 14 — Étude de cas : concevoir et chiffrer

Énoncé : « *ShopFlow* », e-commerce français, 200 000 visiteurs/jour, pic ×5 lors des soldes, données clients RGPD, RTO 1 h / RPO 5 min, budget 6 000 €/mois.

Livrables (voir la grille dans `labs/lab-etude-de-cas/README.md`) :

1. Schéma d'architecture (Mermaid fourni en modèle) avec justification de chaque service.
2. Stratégie DR et tests associés.
3. Estimation de coût via le Pricing Calculator + 5 leviers d'optimisation chiffrés.
4. Revue Well-Architected : 3 risques par pilier et plans d'atténuation.
5. Présentation orale de 10 minutes (jeu de rôle architecte vs client).

Validation : grille d'évaluation sur 100 points (fournie).

Erreurs fréquentes

Erreur	Conséquence
Multi-AZ confondu avec multi-région	Panne régionale non couverte
DR jamais testé	Échec le jour J
Pas de tags	Coûts non attribuables
Sur-ingénierie	Coût et complexité inutiles : l'architecture suit les exigences (RTO/RPO/SLA)
Optimiser sans mesurer	Gains illusoires

Quiz

1. Un RPO de 5 minutes implique quelle caractéristique ?
2. Pilot Light vs Warm Standby ?
3. Quelle première action FinOps sur un compte inconnu ?
4. Quel outil recommande des tailles d'instances ?
5. Pourquoi tester le DR ?

Réponses

1. Réplication continue des données (≤ 5 min de perte), pas de simples sauvegardes quotidiennes.
2. Pilot Light : cœur de données seul actif ; Warm Standby : environnement réduit entièrement fonctionnel.
3. Rendre les coûts visibles : tags, Cost Explorer, budgets, repérer les gaspillages évidents.
4. AWS Compute Optimizer.
5. Pour valider RTO/RPO, runbooks, quotas, permissions et automatisation avant l'incident.

Module 15 — Multi-comptes et gouvernance

Durée : 1 jour · **Niveau** : expert · **Coût** : ≈ 0 € (Organizations gratuit ; Control Tower : coûts des services sous-jacents)
Certifications : SAP, DOP, SCS

Objectifs

- Comprendre pourquoi **un compte = une frontière** (sécurité, coûts, quotas)
- Structurer avec **AWS Organizations**, **OU** et **SCP**
- Concevoir une **landing zone** (Control Tower) et un **accès centralisé** (Identity Center)
- Centraliser logs, sécurité, réseau et coûts

1. Pourquoi plusieurs comptes ?

Un compte AWS est la **frontière d'isolation la plus forte** : identité, facturation, quotas, blast radius. Bonnes pratiques : comptes séparés par **environnement** (dev / staging / prod), par **charge de travail** ou équipe, comptes **transverses** dédiés (sécurité, logs, réseau, outils CI/CD).

2. AWS Organizations

```
Root (management account — ne rien y déployer)
├── OU Security      : audit (lecture), log-archive (logs immuables)
├── OU Infrastructure: network (TGW, DNS), shared-services (CI/CD, ECR, AMI)
├── OU Workloads
│   ├── OU Prod    : app1-prod, app2-prod
│   └── OU NonProd : app1-dev, app1-staging
├── OU Sandbox       : comptes de formation (budget plafonné, nettoyage auto)
└── OU Suspended     : comptes fermés
```

Fonctions : facturation consolidée (remises de volume, partage des Savings Plans), **création programmatique de comptes**, **SCP**, politiques de tags/sauvegarde, **services de confiance** (CloudTrail, Config, GuardDuty, Security Hub, RAM… avec administrateur délégué).

3. Service Control Policies (SCP)

Plafond de permissions pour **tous les principaux d'un compte** (sauf le compte de gestion). Elles **ne donnent aucun droit**, elles limitent. Héritage : une action doit être autorisée à **chaque niveau** (Root → OU → compte).

Exemples de garde-fous :

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "DenyOutsideEU",
      "Effect": "Deny",
      "NotAction": ["iam:*", "organizations:*", "route53:*", "cloudfront:*", "support:*",
"sts:*"],
      "Resource": "*",
      "Condition": { "StringNotEquals": { "aws:RequestedRegion": ["eu-west-3", "eu-west-1",
"eu-central-1"] } }
    },
    {
      "Sid": "ProtectSecurityServices",
      "Effect": "Deny",
      "Action": ["guardduty:DeleteDetector", "cloudtrail:StopLogging",
"cloudtrail:DeleteTrail", "config:StopConfigurationRecorder"],
      "Resource": "*"
    },
    {
      "Sid": "DenyLeaveOrg",
      "Effect": "Deny",
      "Action": "organizations:LeaveOrganization",
      "Resource": "*"
    },
    {
      "Sid": "NoRootUser",
      "Effect": "Deny",
      "Action": "*",
      "Resource": "*",
      "Condition": { "StringLike": { "aws:PrincipalArn": "arn:aws:iam::*:root" } }
    }
  ]
}
```

Stratégies : *deny list* (défaut `FullAWSAccess` + Deny ciblés, recommandé) ou *allow list* (plus strict, plus lourd). **Testez dans une OU sandbox** avant la production.

Autres politiques : **tag policies**, **backup policies**, **AI opt-out**, **declarative policies** (config de base d'un service appliquée à l'organisation).

4. Accès centralisé

IAM Identity Center : un portail, des **permission sets** assignés à des groupes sur des comptes. Intégration IdP (Entra ID, Okta, Google) via SAML/SCIM. Sessions courtes, MFA imposée, pas d'utilisateurs IAM humains.

Accès cross-account pour la CI/CD : le compte *tooling* assume un rôle `DeployRole` dans chaque compte cible (trust vers le compte tooling + conditions OIDC/ExternalId).

5. AWS Control Tower

Met en place une **landing zone** conforme aux bonnes pratiques en quelques clics :

- structure d'OU, comptes *log archive* et *audit*,
- CloudTrail et Config organisationnels,
- **contrôles** (guardrails) préventifs (SCP), détectifs (Config) et proactifs (CloudFormation hooks),
- **Account Factory** (+ AFT : *Account Factory for Terraform*) pour vendre des comptes standardisés en self-service.

6. Services partagés

Besoin	Service
Partager VPC subnets, TGW, règles DNS entre comptes	**AWS RAM**
Réseau hub & spoke	Transit Gateway + Network Firewall + inspection centralisée
Logs centralisés	Organization trail → bucket dans *log-archive* (Object Lock)
Sécurité centralisée	GuardDuty / Security Hub / Config avec administrateur délégué dans *audit*
Facturation	Compte de gestion + Cost Categories + CUR
Déploiement multi-comptes IaC	CloudFormation **StackSets** (organization-aware), Terraform + AFT
Images/artefacts	ECR avec réplication, AMI partagées

7. Modèle opérationnel

- **Compte de gestion** : uniquement facturation et Organizations ; très peu d'accès, MFA matérielle.
- **Break-glass** : rôle d'urgence, alerté à chaque usage.
- **Cycle de vie des comptes** : vente (Account Factory) → baseline (StackSet) → exploitation → fermeture (OU Suspended).
- **Quotas et coûts** par compte, budgets par OU.

8. Démos

Démo 01 — Créer une OU et rattacher un compte : `demos/demo-01-organizations.sh`.
Démo 02 — Effet d'une SCP : attacher `DenyOutsideEU` à l'OU Sandbox, tenter de lancer une instance à `us-east-1` → `explicit deny in a service control policy`.
Démo 03 — Control Tower : parcours de la landing zone et d'Account Factory (lecture).

9. Labo 15 — Gouvernance par SCP (en Terraform)

Prérequis : un compte de gestion Organizations (créez une organisation avec un compte secondaire de test). Si vous n'avez pas ce droit, lisez le code et simulez avec l'IAM Policy Simulator.

Code : `labs/lab-organizations/` — crée une OU `Sandbox`, trois SCP (régions, protection des services de sécurité, interdiction de quitter l'organisation) et les rattache.

Étapes :

1. `terraform apply`.
2. Déplacer un compte de test dans l'OU Sandbox.
3. Depuis ce compte, tenter une action dans `us-east-1` et arrêter CloudTrail : refus attendus.
4. Retirer la SCP régions, constater la levée du blocage.
5. **Défi** : écrire une SCP imposant le tag `owner` à la création d'instances EC2 (`ec2:RunInstances` avec condition `Null` sur `aws:RequestTag/owner`).
6. `terraform destroy`.

Validation : message `explicit deny in a service control policy` observé deux fois.

Erreurs fréquentes

Problème	Cause
SCP « sans effet »	Elles ne s'appliquent pas au compte de gestion ; ou une autre SCP héritée autorise tout
Verrouillage total	SCP trop stricte sur l'OU Root ; toujours tester en sandbox, garder un chemin de secours
Déploiements cross-account refusés	Trust policy ou SCP bloquant `sts:AssumeRole`
Coûts dispersés	Absence de tags et de Cost Categories
Compte de gestion utilisé pour des charges	Risque et blast radius : à interdire

Quiz

1. Une SCP peut-elle donner des permissions ?
2. Quel compte n'est pas soumis aux SCP ?
3. Pourquoi un compte de logs séparé ?
4. Comment industrialiser la création de comptes ?
5. À quoi sert un administrateur délégué ?

Réponses

1. Non, elle ne fait que limiter.
2. Le compte de gestion.
3. Logs immuables et hors de portée d'un compte potentiellement compromis.
4. Control Tower Account Factory / AFT ou Organizations API + baseline en StackSet.
5. Gérer un service organisationnel (GuardDuty, Security Hub…) depuis un compte dédié sans utiliser le compte de gestion.

Module 16 — Projet final : plateforme cloud complète

Durée : 3 jours (individuel ou binôme) · **Niveau** : expert · **Coût** : prévoir 5–15 € si détruit chaque soir
Rôle : évaluation de fin de parcours, et pièce de portfolio.

1. Le sujet

Vous êtes l'ingénieur DevOps de **« TaskHub »**, application de gestion de tâches. Livrez une plateforme **de production** qui héberge l'API et son interface, entièrement **as code**, déployée par **pipeline**, observable, sécurisée, et chiffrée en coût.

Architecture cible

```
Utilisateurs
   |
Route 53  ->  CloudFront + WAF  --(/ statique)-->  S3 front (accès privé, OAC)
                  |
                  +--(/api)-->  ALB (subnets publics)
                                  |
                                ECS Fargate (subnets privés, 2 zones)
                                  |-- RDS PostgreSQL Multi-AZ (subnets data)
                                  |-- SQS --> Worker Lambda
                                  '-- Secrets Manager + KMS

GitHub Actions (OIDC) --> ECR --> ECS      et      Terraform apply
CloudWatch (logs, métriques, alarmes) --> SNS
CloudTrail / Config / GuardDuty --> S3 logs
```

2. Exigences fonctionnelles et techniques

Obligatoires (60 pts)

#	Exigence
1	**Réseau** : VPC 2 AZ, subnets public/privé/data, aucun accès Internet entrant direct vers app et BD, endpoints S3 (et ECR si NAT absente)
2	**Compute** : API conteneurisée sur ECS Fargate (≥ 2 tâches, 2 AZ), image dans ECR (tags = SHA, scan activé)
3	**Données** : RDS PostgreSQL chiffré, Multi-AZ, sauvegardes, secret géré (Secrets Manager)
4	**Edge** : CloudFront + S3 privé (OAC) pour le front ; route `/api/*` vers l'ALB ; HTTPS
5	**IaC** : 100 % Terraform (modules + backend S3 distant), `fmt/validate/tflint/checkov` verts
6	**CI/CD** : GitHub Actions + OIDC (aucune clé), build → test → scan → push ECR → déploiement ECS avec rollback automatique ; plan Terraform sur PR

#	Exigence
7	**Observabilité** : dashboard, alarmes 5xx/latence/CPU/BD, logs avec rétention, SNS vers e-mail
8	**Sécurité** : moindre privilège (rôles dédiés), WAF managé, IMDSv2, pas de secret en clair, chiffrement KMS
9	**Coûts** : tags obligatoires, budget, estimation mensuelle documentée
10	**Documentation** : README, diagramme, runbook d'exploitation, ADR (décisions d'architecture)

Bonus (jusqu'à 25 pts)

- Blue/Green avec CodeDeploy, ou canary (+5)
- Multi-environnements dev/prod avec comptes séparés (Organizations) (+5)
- Autoscaling par requêtes + test de charge documenté (+4)
- DR : réplica inter-région + procédure testée (+5)
- Traçage X-Ray/OpenTelemetry (+3)
- Tests d'infra Terratest / `terraform test` (+3)

3. Planning conseillé

Jour	Objectifs	Résultat attendu
J1 – Fondations	Backend d'état, réseau, ECR, RDS, secrets	`terraform apply` sur l'infra de base
J2 – Application & pipeline	ECS + ALB, CloudFront/S3, OIDC, workflow CI/CD	Une modification du code se déploie automatiquement
J3 – Production-ready	Observabilité, WAF, sécurité, rollback, coût, doc, démo	Soutenance

Un squelette de démarrage est fourni dans `starter/` (structure de dossiers, `Makefile`, ADR modèle, check-list).

4. Livrables

1. Dépôt Git (code, pipelines, doc).
2. `ARCHITECTURE.md` (schéma + décisions + alternatives écartées).
3. `RUNBOOK.md` : déploiement, rollback, incident « 5xx en hausse », restauration de BD, rotation de secret.
4. `COSTS.md` : estimation mensuelle, 5 optimisations.
5. **Démo live de 15 min** : push de code → pipeline → déploiement ; panne simulée (tuer une tâche, forcer le failover RDS) ; alerte reçue ; revue de sécurité (Security Hub / checkov).

5. Grille d'évaluation (100 pts + bonus)

Domaine	Pts	Critères
Architecture & réseau	15	Isolation correcte, multi-AZ, justification
IaC	15	Modularité, state distant, lisibilité, absence de duplication
CI/CD	15	OIDC, étapes, scans, rollback, promotion d'artefact
Sécurité	15	Moindre privilège, chiffrement, secrets, WAF, durcissement
Observabilité & exploitation	10	Alarmes pertinentes, runbook, logs exploitables
Fiabilité	10	Résistance démontrée aux pannes
Coûts	5	Tags, estimation, optimisations
Documentation & soutenance	15	Clarté, ADR, démonstration

Seuils : < 50 à retravailler · 50-69 acquis · 70-84 solide · ≥ 85 **expert**.

6. Pièges à éviter

- Commencer par l'application plutôt que par les fondations (réseau, état, pipeline).
- Tout mettre dans un seul état Terraform géant (découper : `network`, `data`, `app`).
- Utiliser des clés d'accès « temporairement » (elles restent).
- Oublier `terraform destroy` entre deux sessions (💰 NAT, ALB, RDS Multi-AZ).
- Ne pas tester les restaurations.

7. Auto-évaluation finale

Cochez uniquement ce que vous savez faire **sans aide** :

- ☐ Concevoir un VPC multi-AZ et expliquer chaque route
- ☐ Écrire une policy IAM au moindre privilège avec conditions
- ☐ Déployer ECS/Fargate derrière ALB avec autoscaling et rollback
- ☐ Industrialiser avec Terraform (modules, backend, CI) et diagnostiquer une dérive
- ☐ Construire un pipeline OIDC sans clé
- ☐ Définir alarmes, dashboards et SLO utiles
- ☐ Chiffrer avec KMS et gérer des secrets avec rotation
- ☐ Dimensionner un DR selon RTO/RPO
- ☐ Gouverner un parc de comptes avec SCP

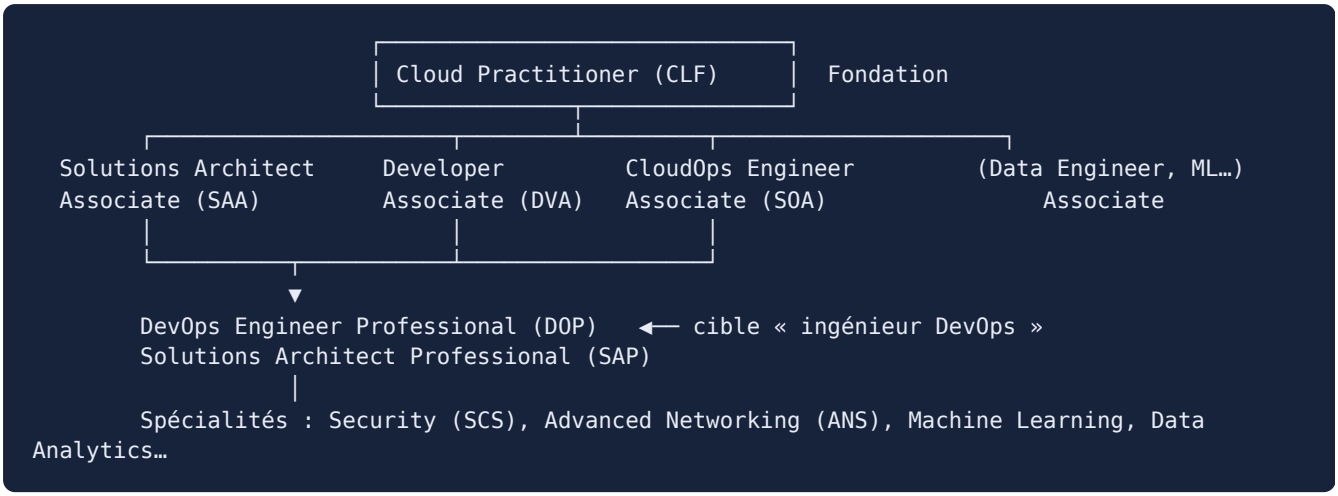
- [ ] Optimiser les coûts d'une architecture existante

Module 17 — Préparation aux certifications AWS

Durée : 2 à 10 semaines selon la certification · **Format** : révision guidée + questions + examens blancs

> Les codes, durées, prix et pondérations d'examen évoluent. **Vérifiez toujours le guide d'examen officiel** (« Exam Guide » PDF) sur aws.amazon.com/certification avant de planifier.

1. Le parcours de certification



Que passer, dans quel ordre ?

Votre profil	Recommandation
Débutant total / non technique	CLF → SAA
Dev ou DevOps	**SAA → DVA → DOP** (le SAA donne la vision d'ensemble)
Ops / SRE / admin	SAA → SOA → DOP
Architecte	SAA → SAP → spécialités
Sécurité	SAA → SCS

Les associate sont valables 3 ans ; la certification d'un niveau supérieur **renouvelle** les inférieures du même domaine selon les règles en vigueur — à vérifier.

2. Format général des examens

- **QCM à choix unique** et **à choix multiples** (« choisissez 2/3 »), des questions de type ordre/appariement possibles.

- Associate : ~65 questions, ~130 min. Professional : ~75 questions, ~180 min. Fondation : ~65 questions, 90 min.
- Score de 100 à 1000, seuil de réussite 700 (CLF/associate) ou 750 (professional/spécialités).
- Certaines questions ne sont **pas notées** (items de test).
- Passage en centre Pearson VUE ou en ligne surveillé ; 30 min supplémentaires possibles pour les non-natifs anglophones (demande préalable) ; examens disponibles en plusieurs langues dont le français.

3. Domaines et pondérations (à confirmer avec le guide officiel)

CLF-C02 — Cloud Practitioner

Domaine	%
Concepts du cloud	24
Sécurité et conformité	30
Technologie et services cloud	34
Facturation, tarification, support	12

SAA-C03 — Solutions Architect Associate

Domaine	%
Architectures **sécurisées**	30
Architectures **résilientes**	26
Architectures **performantes**	24
Architectures **optimisées en coût**	20

DVA-C02 — Developer Associate

Domaine	%
Développement avec les services AWS	32
Sécurité	26
Déploiement	24
Dépannage et optimisation	18

DOP-C02 — DevOps Engineer Professional

Domaine	%
Automatisation du SDLC (CI/CD)	22

Domaine	%
Gestion de configuration et IaC	17
Solutions résilientes	15
Surveillance et journalisation	15
Réponse aux incidents et événements	14
Sécurité et conformité	17

SOA (CloudOps Engineer) — Associate

Surveillance/observabilité, fiabilité et continuité, déploiement/automatisation, sécurité et conformité, réseau, coûts et performances. **Consultez le guide à jour** : l'examen a été renommé et remanié, il comporte des labs pratiques dans certaines versions.

4. Correspondance modules ↔ examens

Module	CLF	SAA	DVA	SOA	DOP
01 Fondamentaux	●●●	●	●	●	
02 IAM	●●	●●●	●●●	●●●	●●●
03 VPC	●	●●●	●	●●●	●●
04 EC2	●●	●●●	●●	●●●	●●
05 Stockage	●●	●●●	●●	●●●	●●
06 BD	●	●●●	●●●	●●	●●
07 HA & edge	●	●●●	●●	●●●	●●●
08 Serverless	●	●●●	●●●	●●	●●●
09 Conteneurs	●	●●	●●	●●	●●●
10 IaC	●	●	●●	●●●	●●●
11 CI/CD		●	●●●	●●	●●●
12 Observabilité	●	●●	●●	●●●	●●●
13 Sécurité	●●●	●●●	●●●	●●●	●●●
14 Architecture/FinOps	●●●	●●●	●	●●	●●
15 Multi-comptes	●	●●		●●	●●●

5. Plans de révision

Hypothèse : 8-10 h/semaine. Chaque semaine : cours + labos du module + 25 questions + flashcards.

SAA-C03 — 8 semaines

Sem.	Contenu
1	Modules 01-02 (cloud, IAM)
2	Module 03 (VPC)
3	Modules 04-05 (EC2, stockage)
4	Modules 06-07 (BD, HA, edge)
5	Module 08 (serverless, messaging) + conteneurs (aperçu)
6	Module 13 (sécurité) + 12 (monitoring)
7	Module 14 (Well-Architected, DR, coûts) + analytique/migration (aperçu)
8	2 examens blancs complets + revue des erreurs

DOP-C02 — 10 semaines (après SAA ou expérience équivalente)

Sem.	Contenu
1	IaC 1 : CloudFormation avancé (change sets, StackSets, custom resources, helper scripts, `cfn-signal`)
2	IaC 2 : CDK/SAM, Terraform ; Service Catalog
3	CI/CD 1 : CodeBuild, CodePipeline, artefacts, approbations, tests
4	CI/CD 2 : stratégies de déploiement (EC2, Lambda, ECS), Beanstalk, rollback
5	Conteneurs ECS/EKS, ECR, déploiements
6	Observabilité : CloudWatch avancé, X-Ray, Logs Insights, agrégation multi-comptes
7	Réponse aux incidents : EventBridge, SSM Automation, Config remediation, Systems Manager
8	Sécurité & conformité : IAM avancé, KMS, Secrets, GuardDuty, Security Hub, Control Tower, SCP
9	Résilience : DR, multi-région, Route 53 (failover, latence), Aurora Global, ASG avancé
10	3 examens blancs, lecture des FAQ et whitepapers clés

CLF-C02 — 2 à 3 semaines

Semaine 1 : modules 01, 02, 13 · Semaine 2 : survol des services (04-08) et modèle de tarification · Semaine 3 : examens blancs.

DVA-C02 — 6 semaines

Priorité : modules 02, 06 (DynamoDB), 08 (Lambda, API GW, SQS/SNS), 10 (SAM/CloudFormation), 11 (CI/CD), 12 (X-Ray, CloudWatch), 13 (KMS, Cognito, Secrets).

6. Méthode de travail efficace

1. **Pratiquer d'abord, mémoriser ensuite** : chaque service doit avoir été manipulé (labos de ce dépôt).
2. **Lire le guide d'examen** et cocher chaque compétence.
3. **Réviser en espaçant** : flashcards (Anki) → flashcards.md.
4. **Examens blancs** : au moins 3 ; analyser *toutes* les réponses, y compris les justes.
5. **Cahier d'erreurs** : une ligne par erreur, concept manquant + règle de décision.
6. **Whitepapers & docs** : Well-Architected Framework, FAQ des services majeurs (S3, EC2, VPC, IAM, RDS, DynamoDB, Lambda).
7. **Écrire des schémas** de mémoire (VPC 3 niveaux, pipeline, DR).
8. Passer l'examen quand vous obtenez **≥ 80 %** régulièrement sur des examens blancs de qualité.

7. Stratégie le jour de l'examen

- **Lisez la dernière phrase en premier** : « le plus économique », « le moins d'effort opérationnel », « la plus haute disponibilité »… elle fixe le critère.
- **Mots-clés** → services (voir cheatsheet.md).
- **Éliminez** d'abord les réponses fausses : techniquement impossibles, trop complexes, sur-dimensionnées.
- AWS préfère : **managé > auto-géré**, **serverless > serveurs**, **découplé > couplé**, **moindre privilège**, **rôle > clé d'accès**, **chiffrement partout**, **multi-AZ**.
- Questions « choisissez 2 » : chaque option doit être *correcte* **et** répondre à la question.
- Marquez les questions longues, **ne bloquez pas** plus de 2 min en première passe (≈ 1,8 min/question en moyenne ; ≈ 2,4 min en professional).
- Gardez 10-15 min pour relire les questions marquées.
- Pas de pénalité pour les mauvaises réponses : **répondez toujours**.

8. Ressources

Type	Ressource
Officiel	AWS Skill Builder (cours gratuits, Exam Prep, **examens blancs officiels**), Exam Guides, Sample Questions
Docs	Documentation, FAQ de chaque service, Well-Architected, *Architecture Center*
Pratique	AWS Workshops (workshops.aws), Well-Architected Labs, ce dépôt
Communauté	re:Post, Reddit r/AWSCertifications, GitHub « aws-samples »

Type	Ressource
Avancé	re:Invent (vidéos « deep dive » : VPC, IAM, S3, DynamoDB, Lambda)

9. Contenu de ce module

Fichier	Rôle
questions-entrainement.md	45 questions d'examen commentées par domaine
flashcards.md	134 cartes question/réponse (import Anki possible)
cheatsheet.md	Aide-mémoire : mots-clés → service, limites, comparatifs
suivi-progression.md	Tableau de suivi et cahier d'erreurs

10. Évaluation de fin de module

- Examen blanc cible : **≥ 80 %** sur 3 tentatives de qualité.
- Quiz de fin de chaque module (≥ 90 %).
- Projet final (module 16) noté ≥ 70.

Bon courage — et n'oubliez pas : **la certification valide des compétences, la pratique les construit.**

Questions d'entraînement (style examen)

45 questions de niveau Associate → Professional. Répondez d'abord, puis ouvrez la correction. Format : `[SAA]`, `[DVA]`, `[SOA]`, `[DOP]` = certification la plus concernée. « Choisissez 2 » = deux réponses.

Domaine A — Sécurité et IAM

Q1 [SAA] Une application sur EC2 doit lire des objets d'un bucket S3. Quelle est la solution la plus sécurisée ?

- A. Stocker des clés d'accès IAM dans un fichier de l'instance
- B. Attacher à l'instance un rôle IAM via un profil d'instance avec les permissions S3 minimales
- C. Rendre le bucket public en lecture
- D. Coder les clés dans les variables d'environnement de l'AMI

Réponse

B. Les rôles fournissent des credentials temporaires renouvelés automatiquement via IMDS ; aucun secret à stocker. A/D exposent des clés longue durée, C expose les données.

Q2 [SAA] Un utilisateur a une policy IAM `Allow s3:*` mais une SCP de l'organisation refuse `s3:DeleteBucket`. Peut-il supprimer un bucket ?

- A. Oui, l'Allow IAM prime
- B. Non, un Deny (ou l'absence d'Allow) dans la SCP plafonne les permissions
- C. Oui, sauf si le bucket est versionné
- D. Seulement avec le compte root

Réponse

B. Les SCP définissent le maximum autorisé ; un Deny explicite l'emporte toujours. Les SCP s'appliquent aussi aux utilisateurs IAM du compte membre (y compris, pour l'essentiel, root du compte membre).

Q3 [SAA] Un partenaire dans un autre compte AWS doit accéder à un bucket S3 de façon sûre. Quelle approche respecte le moindre privilège ?

- A. Créer un utilisateur IAM avec clés pour le partenaire
- B. Un rôle IAM assumable par le compte du partenaire, avec un `ExternalId`
- C. Partager les identifiants root
- D. Bucket public

Réponse

B. Rôle cross-account + trust policy ; l'ExternalId protège contre le *confused deputy*. Alternative : policy de bucket nommant le rôle du partenaire.

Q4 [SAA] Les données S3 doivent être chiffrées avec une clé dont vous contrôlez la rotation et pour laquelle chaque usage est audité dans CloudTrail. Que choisir ?

- A. SSE-S3
- B. SSE-KMS avec une clé gérée par le client
- C. SSE-C
- D. Pas de chiffrement, TLS seulement

Réponse

B. SSE-KMS (CMK) : contrôle de la policy de clé, rotation et journalisation de chaque appel `Decrypt/GenerateDataKey`. SSE-S3 n'offre pas ce contrôle/audit.

Q5 [DVA] Une Lambda lit un secret dans Secrets Manager chiffré avec une CMK. Elle obtient `AccessDeniedException` bien que le rôle ait `secretsmanager:GetSecretValue`. Que manque-t-il ?

- A. `kms:Decrypt` sur la clé (policy de clé ou IAM)
- B. `s3:GetObject`
- C. Un VPC endpoint obligatoire
- D. Le rôle doit être root

Réponse

A. Pour déchiffrer le secret il faut aussi l'autorisation sur la clé KMS (et la policy de clé doit permettre l'accès).

Q6 [SAA] Vous devez bloquer toute requête HTTP non chiffrée vers un bucket. Quel mécanisme ?

- A. NACL
- B. Policy de bucket avec `Deny` si `aws:SecureTransport` est `false`
- C. Security Group
- D. Un rôle IAM

Réponse

B.

Q7 [SOA] Quel outil identifie les ressources partagées avec une entité externe à l'organisation ?

- A. GuardDuty
- B. IAM Access Analyzer
- C. Inspector
- D. Trusted Advisor uniquement

Réponse

B. Access Analyzer analyse les policies de ressource (S3, KMS, rôles, SQS, Lambda...) pour détecter l'accès externe.

Q8 [DOP] GitHub Actions doit déployer dans AWS sans stocker de clé. Quelle architecture ?

- A. Utilisateur IAM + clés dans les secrets GitHub
- B. Fournisseur OIDC IAM + rôle avec trust policy filtrée sur `sub`, assumé avec `AssumeRoleWithWebIdentity`
- C. Clés root
- D. Un bucket S3 public avec les clés

Réponse

B.

Q9 [SAA] (choisissez 2) Quelles mesures protègent un compte root ?

- A. Activer le MFA
- B. Créer des clés d'accès pour l'automatiser
- C. L'utiliser uniquement pour les tâches qui l'exigent
- D. Partager son mot de passe avec l'équipe
- E. Désactiver CloudTrail

Réponse

A et C.

Domaine B — Réseau et connectivité

Q10 [SAA] Des instances dans un subnet privé doivent télécharger des correctifs depuis Internet, sans être joignables depuis Internet. Quelle solution ?

- A. Internet Gateway dans le subnet privé
- B. NAT Gateway dans un subnet public + route `0.0.0.0/0` du subnet privé vers elle
- C. IP publiques sur les instances

- D. VPC peering

Réponse

B.

Q11 [SAA] Le trafic vers S3 depuis un VPC génère de forts coûts de NAT. Comment le réduire sans exposer le trafic à Internet ?

- A. Endpoint de passerelle (Gateway endpoint) S3
- B. Transit Gateway
- C. Direct Connect
- D. Accélération de transfert S3

Réponse

A. Gratuit, via la table de routage, le trafic reste sur le réseau AWS.

Q12 [SAA] Quelle différence principale entre un Security Group et une NACL ?

- A. SG stateless, NACL stateful
- B. SG stateful et au niveau instance (Allow seulement) ; NACL stateless et au niveau subnet (Allow/Deny)
- C. Aucune
- D. NACL s'applique à une région entière

Réponse

B.

Q13 [SAA] Dix VPC dans plusieurs comptes et un site on-premises doivent communiquer, avec routage transitif. Quel service ?

- A. VPC Peering en maillage
- B. AWS Transit Gateway
- C. NAT Gateway
- D. PrivateLink pour chaque VPC

Réponse

B. Hub & spoke transitif ; le peering n'est pas transitif et devient ingérable en maillage.

Q14 [SOA] Une instance dans un subnet public ne répond pas à Internet. Les SG sont corrects. Que vérifier en premier ? (choisissez 2)

- A. Présence d'une route `0.0.0.0/0` vers l'Internet Gateway
- B. L'instance a-t-elle une IP publique/EIP ?
- C. La version du noyau
- D. Le type d'instance
- E. La taille du volume

Réponse

A et B. (puis NACL).

Q15 [SAA] Exposer un service interne (derrière un NLB) à des centaines de VPC clients, sans chevauchement d'adresses ni peering. Que choisir ?

- A. AWS PrivateLink (endpoint service)
- B. VPC Peering
- C. Internet Gateway
- D. Site-to-Site VPN

Réponse

A.

Q16 [SAA] Comment obtenir des adresses IP statiques d'entrée pour une application TCP multi-région avec bascule rapide ?

- A. ALB
- B. AWS Global Accelerator
- C. Route 53 simple
- D. CloudFront

Réponse

B. IP anycast statiques, bascule en secondes, TCP/UDP.

Domaine C — Compute, scalabilité et haute disponibilité

Q17 [SAA] Une charge batch tolérante aux interruptions doit minimiser le coût. Quelle option ?

- A. On-Demand
- B. Instances Spot
- C. Dedicated Hosts
- D. Reserved Instances 3 ans

Réponse

B.

Q18 [SAA] Une application web doit survivre à la perte d'une AZ et absorber des pics de trafic HTTP. Quelle architecture ?

- A. Une grosse instance dans une AZ
- B. ALB + Auto Scaling Group sur ≥ 2 AZ avec scaling target tracking
- C. Une instance avec EIP
- D. NLB et une instance

Réponse

B.

Q19 [SAA] Les instances d'un ASG sont terminées puis recréées en boucle. Quelle est la cause la plus probable ?

- A. Le health check ELB échoue (chemin, SG ou démarrage trop lent vs grace period)
- B. Les AMI sont chiffrées
- C. Le VPC est trop grand
- D. Le type d'instance est trop récent

Réponse

A.

Q20 [SAA] Quelle option améliore la **disponibilité** d'une base RDS ? Et laquelle améliore les **performances en lecture** ?

- A. Multi-AZ / Read Replica
- B. Read Replica / Multi-AZ
- C. Multi-AZ pour les deux
- D. Snapshots / Multi-AZ

Réponse

A. Multi-AZ = disponibilité (standby synchrone) ; Read Replica = lecture (asynchrone).

Q21 [DVA] Des milliers de Lambdas saturent les connexions d'une base RDS. Que faire ?

- A. Augmenter la mémoire Lambda
- B. Utiliser RDS Proxy

- C. Supprimer le VPC
- D. Passer en Multi-AZ

Réponse

B.

Q22 [DOP] Vous devez déployer une nouvelle AMI sur un ASG sans interruption et avec retour arrière simple. Quelle approche ?

- A. Modifier les instances en SSH
- B. Nouveau Launch Template + Instance Refresh (rolling) avec min healthy %, ou déploiement blue/green avec un second ASG
- C. Terminer toutes les instances
- D. Redémarrer l'ASG

Réponse

B.

Q23 [SAA] Un site statique mondial doit être rapide, en HTTPS, avec un bucket S3 non public. Que choisir ?

- A. Static website hosting public S3
- B. CloudFront avec OAC vers le bucket privé, certificat ACM (us-east-1)
- C. EC2 nginx
- D. Route 53 uniquement

Réponse

B.

Q24 [SAA] Lors d'une panne du site principal, Route 53 doit rediriger vers un site de secours. Quelle politique ?

- A. Pondérée sans health check
- B. Failover avec health check
- C. Simple
- D. Géolocalisation

Réponse

B.

Domaine D — Stockage et bases de données

Q25 [SAA] Des données dont le profil d'accès est imprévisible doivent minimiser le coût sans gestion manuelle. Quelle classe S3 ?

- A. Standard
- B. Intelligent-Tiering
- C. Glacier Deep Archive
- D. One Zone-IA

Réponse

B.

Q26 [SAA] Un fichier S3 versionné a été « supprimé » par erreur. Comment le récupérer ?

- A. Impossible
- B. Supprimer le delete marker (ou restaurer une version précédente)
- C. Restaurer depuis EBS
- D. Contacter le support

Réponse

B.

Q27 [SAA] Plusieurs instances Linux dans différentes AZ doivent partager un système de fichiers POSIX. Quel service ?

- A. EBS
- B. EFS
- C. Instance store
- D. S3 Glacier

Réponse

B.

Q28 [DVA] Une table DynamoDB subit du throttling sur une seule valeur de partition key. Quelle est la cause ? Comment y remédier ?

- A. Hot partition : mieux répartir la clé (suffixe/sharding), ou passer en on-demand/DAX pour les lectures
- B. Le TTL est actif
- C. Les index sont absents

- D. La région est trop lointaine

Réponse

A.

Q29 [DVA] Vous devez interroger des commandes par `statut`, alors que la clé primaire est `clientId`. Que faire ?

- A. `Scan` avec filtre
- B. Créer un Global Secondary Index sur `statut`
- C. Dupliquer la table
- D. Ajouter un LSI sur `statut`

Réponse

B. Le LSI doit partager la partition key et se crée à la création de la table ; ici GSI.

Q30 [SAA] Besoin d'un RPO ≈ 1 s et d'un failover régional pour une base MySQL-compatible. Quelle solution ?

- A. RDS Multi-AZ
- B. Aurora Global Database
- C. Snapshots quotidiens
- D. Read Replica dans la même AZ

Réponse

B.

Q31 [SAA] Un cache en mémoire pour réduire la charge de lecture d'une base relationnelle ?

- A. ElastiCache
- B. S3
- C. Glacier
- D. Redshift

Réponse

A.

Domaine E — Serverless, intégration et messaging

Q32 [DVA] Une file SQS reçoit des messages retraités à répétition alors que la Lambda réussit. Cause probable ?

- A. Visibility timeout inférieur à la durée de traitement
- B. Long polling activé
- C. Le message dépasse 256 Ko
- D. Le DLQ est vide

Réponse

A.

Q33 [SAA] Un message doit déclencher trois traitements indépendants et fiables. Quelle conception ?

- A. Une file SQS unique avec trois consommateurs
- B. Topic SNS avec trois files SQS abonnées (fan-out)
- C. Trois appels synchrones
- D. Une seule Lambda séquentielle

Réponse

B.

Q34 [DVA] Comment isoler les messages qui échouent après 3 tentatives ?

- A. Augmenter le timeout
- B. Dead-letter queue avec `maxReceiveCount=3`
- C. Supprimer le message
- D. FIFO uniquement

Réponse

B.

Q35 [DVA] Quelle est la durée maximale d'exécution d'une Lambda ?

- A. 5 min B. 15 min C. 1 h D. 24 h

Réponse

B. Pour plus long : Step Functions, ECS/Fargate, Batch.

Q36 [SAA] Un workflow multi-étapes avec attente, parallélisme et reprise sur erreur : quel service ?

- A. Step Functions
- B. SQS
- C. SNS
- D. CloudFront

Réponse

A.

Q37 [DVA] Quelle intégration réduit le cold start d'une API critique en Java ?

- A. SnapStart ou concurrence provisionnée
- B. Réduire le timeout
- C. Utiliser x86 uniquement
- D. Désactiver les logs

Réponse

A.

Domaine F — IaC, CI/CD, déploiement

Q38 [DOP] Comment prévisualiser l'impact d'une mise à jour CloudFormation en production ?

- A. Exécuter directement
- B. Créer un change set et l'examiner avant exécution
- C. Supprimer la stack
- D. Drift detection

Réponse

B.

Q39 [DOP] Une ressource a été modifiée manuellement hors CloudFormation. Quelle fonctionnalité le détecte ?

- A. Change set
- B. Drift detection
- C. Stack policy
- D. Rollback trigger

Réponse

B.

Q40 [DOP] Un déploiement ECS doit basculer 10 % du trafic, valider, puis le reste, avec rollback automatique sur alarme. Quel service ?

- A. CodeDeploy blue/green avec configuration canary et alarmes CloudWatch
- B. Rolling update sans alarme
- C. Redémarrage manuel
- D. CodeCommit

Réponse

A.

Q41 [DOP] Déployer une même baseline (rôle, Config, GuardDuty) dans tous les comptes et régions d'une organisation, y compris les futurs comptes. Quelle solution ?

- A. CloudFormation StackSets avec déploiement géré par les services (organization-aware, auto-deployment)
- B. Script manuel
- C. Une stack par console
- D. Elastic Beanstalk

Réponse

A.

Q42 [DOP] Le state Terraform est partagé par une équipe. Comment éviter les `apply` concurrents et la perte de données ?

- A. State local sur un poste
- B. Backend S3 versionné et chiffré avec verrouillage (DynamoDB ou lock S3)
- C. Commit du state dans Git
- D. Aucun contrôle

Réponse

B.

Domaine G — Observabilité, opérations, résilience, coûts

Q43 [SOA] Comment obtenir la mémoire utilisée par une instance EC2 dans CloudWatch ?

- A. Métrique native EC2
- B. Installer l'agent CloudWatch et collecter les métriques personnalisées
- C. Cost Explorer
- D. CloudTrail

Réponse

B.

Q44 [DOP] À chaque arrêt d'une instance EC2 de production, une alerte doit être envoyée et un ticket créé automatiquement. Comment ?

- A. Alarme CPU
- B. Règle EventBridge sur « EC2 Instance State-change Notification » (stopped) vers SNS/Lambda
- C. Lecture manuelle des logs
- D. Config uniquement

Réponse

B.

Q45 [SAA] (RTO 30 min, RPO 5 min, budget limité) Quelle stratégie de DR ?

- A. Backup & Restore
- B. Pilot Light ou Warm Standby avec réplication continue des données
- C. Multi-site actif-actif
- D. Aucune

Réponse

B. Backup & Restore ne tient pas le RTO ; l'actif-actif est surdimensionné (coût).

Grille d'auto-évaluation

Score	Niveau
< 60 %	Retravailler les modules concernés
60-79 %	Bon ; cibler les domaines faibles

Score	Niveau
≥ 80 %	Prêt pour un examen blanc complet

Notez vos erreurs dans suivi-progression.md.

Flashcards

Format `Question ;; Réponse` (une par ligne) → importable dans Anki (séparateur `;;` à configurer).

Fondamentaux

- **Qu'est-ce qu'une Région AWS** : Zone géographique isolée contenant plusieurs AZ
- **Qu'est-ce qu'une AZ** : Un ou plusieurs datacenters isolés dans une région, reliés par réseau à faible latence
- **Responsabilité partagée : qui patche l'OS d'une EC2** : Le client
- **Responsabilité partagée : qui gère l'hôte physique** : AWS
- **Le trafic entrant depuis Internet est-il facturé** : Non
- **Modèle d'achat le moins cher pour charge interruptible** : Spot (jusqu'à -90 %)
- **Engagement 1-3 ans flexible sur le calcul** : Compute Savings Plans
- **Outil pour estimer un coût avant déploiement** : Pricing Calculator
- **Outil pour visualiser et analyser les coûts passés** : Cost Explorer
- **Les 6 piliers Well-Architected** : Excellence opérationnelle, Sécurité, Fiabilité, Performance, Coûts, Durabilité

IAM

- **Deny explicite vs Allow** : Le Deny explicite l'emporte toujours
- **Par défaut une requête est** : Refusée (deny implicite)
- **Rôle vs utilisateur** : Le rôle donne des credentials temporaires, sans mot de passe, assumable
- **Trust policy** : Définit qui peut assumer le rôle
- **Permission boundary** : Plafond de permissions d'un user/rôle (ne donne aucun droit)
- **SCP** : Plafond de permissions au niveau compte/OU (ne donne aucun droit)
- **Accès cross-account S3 avec policy de ressource** : Allow requis côté identité ET côté ressource
- **Condition pour forcer le MFA** : aws:MultiFactorAuthPresent = true
- **Service pour SSO multi-comptes** : IAM Identity Center
- **Service qui génère des policies depuis l'activité** : IAM Access Analyzer (policy generation)
- **Credentials d'une EC2 sans clé** : Rôle via instance profile (IMDS)
- **Protection contre le confused deputy** : ExternalId / aws:SourceArn
- **GitHub Actions sans clé** : OIDC + AssumeRoleWithWebIdentity

Réseau

- **Un subnet s'étend sur** : Une seule AZ
- **IP réservées par subnet** : 5
- **Subnet public =** : Route 0.0.0.0/0 vers une Internet Gateway

- **Sortie Internet d'un subnet privé** : NAT Gateway (dans un subnet public)
- **Security Group** : Stateful, niveau instance, Allow uniquement
- **NACL** : Stateless, niveau subnet, Allow et Deny, règles ordonnées
- **Endpoint gratuit** : Gateway endpoint (S3, DynamoDB)
- **PrivateLink** : Exposer un service via endpoint d'interface sans peering
- **Peering est-il transitif** : Non
- **Hub & spoke multi-VPC** : Transit Gateway
- **VPN site-to-site : nombre de tunnels** : 2 (IPsec)
- **Lien dédié on-premises** : Direct Connect
- **Diagnostic du trafic accepté/refusé** : VPC Flow Logs
- **Analyse statique de connectivité** : Reachability Analyzer

Compute

- **EC2 burstable** : Famille T (crédits CPU)
- **Instance Graviton** : Suffixe g (ARM), ~20 % moins cher
- **IMDSv2** : Jeton de session obligatoire, protège du SSRF
- **Accès shell sans port SSH** : SSM Session Manager
- **Instance store à l'arrêt** : Données perdues
- **AMI est** : Régionale (copiable entre régions)
- **Launch Configuration vs Template** : Les Configurations sont obsolètes, utiliser les Templates
- **Placement group cluster / spread / partition** : Latence mini / isolation matérielle / gros clusters distribués
- **Politique de scaling recommandée** : Target tracking

Stockage

- **Durabilité S3** : 11 neuf
- **Classe S3 pour profil inconnu** : Intelligent-Tiering
- **Archive la moins chère** : Glacier Deep Archive
- **Protéger contre suppression accidentelle S3** : Versioning (+ MFA Delete, Object Lock)
- **WORM S3** : Object Lock
- **Accès temporaire à un objet privé** : Presigned URL
- **Réplication S3 requiert** : Versioning sur source et destination
- **Bloc partagé multi-AZ** : Non : EBS est lié à une AZ
- **Système de fichiers partagé Linux** : EFS
- **Système de fichiers Windows** : FSx for Windows File Server
- **Sauvegarde centralisée** : AWS Backup
- **Type EBS par défaut recommandé** : gp3
- **Snapshots EBS** : Incrémentaux, stockés dans S3, régionaux

Bases de données

- **Multi-AZ RDS** : HA/DR, réplication synchrone, standby non lisible
- **Read Replica** : Performance lecture, asynchrone, inter-région possible
- **RDS Proxy** : Pool de connexions, utile avec Lambda
- **Aurora : copies de données** : 6 copies sur 3 AZ
- **Aurora multi-région** : Aurora Global Database (RPO ~1 s)
- **Chiffrer une RDS existante non chiffrée** : Snapshot → copie chiffrée → restauration
- **DynamoDB : clé primaire** : Partition key (+ sort key optionnelle)
- **GSI vs LSI** : GSI: autre PK, créé à tout moment ; LSI: même PK, à la création de la table
- **Éviter** : Scan sur grandes tables
- **Cache DynamoDB microseconde** : DAX
- **DynamoDB multi-région actif-actif** : Global Tables
- **Expiration automatique d'items** : TTL
- **Cache en mémoire** : ElastiCache (Redis/Valkey, Memcached)
- **Entrepôt de données** : Redshift
- **Migration de BD avec peu d'arrêt** : DMS (+ SCT si hétérogène)

HA et edge

- **ALB vs NLB** : ALB: L7 HTTP, routage chemin/hôte ; NLB: L4, IP statique, très faible latence
- **Certificat TLS gratuit** : ACM
- **Certificat pour CloudFront** : ACM en us-east-1
- **Alias vs CNAME** : Alias: apex autorisé, gratuit, cible AWS
- **Routage Route 53 pour basculement** : Failover avec health check
- **CDN** : CloudFront
- **Protéger S3 derrière CloudFront** : OAC (Origin Access Control)
- **IP anycast statiques** : Global Accelerator
- **Protection DDoS L3/L4 gratuite** : Shield Standard
- **Pare-feu L7** : WAF

Serverless et messaging

- **Durée max Lambda** : 15 minutes
- **Concurrence réservée vs provisionnée** : Plafond/garantie vs instances pré-chauffées (pas de cold start)
- **Lambda asynchrone : réessais** : 2, puis DLQ/destination
- **SQS Standard vs FIFO** : At-least-once non ordonné vs ordre + exactly-once (300 msg/s)
- **Visibility timeout** : Doit dépasser la durée de traitement
- **Taille max message SQS** : 256 Ko
- **SNS** : Pub/Sub push, fan-out
- **EventBridge** : Bus d'événements, règles, Scheduler

- **Orchestration de workflow** : Step Functions
- **Flux ordonné rejouable** : Kinesis Data Streams
- **API Gateway HTTP vs REST** : HTTP: moins cher/simple ; REST: clés d'API, plans d'usage, cache, validation

Conteneurs

- **Task role vs execution role** : Task: code de l'appli ; Execution: pull ECR, logs, secrets
- **Mode réseau Fargate** : awsvpc
- **Serveurs à gérer sur Fargate** : Aucun
- **Accès AWS pour un pod EKS** : IRSA ou Pod Identity
- **Registre d'images** : ECR
- **Rollback auto ECS** : Deployment circuit breaker / CodeDeploy
- **Shell dans un conteneur Fargate** : ECS Exec

IaC et CI/CD

- **Prévisualiser CloudFormation** : Change set
- **Détecter les modifications manuelles** : Drift detection
- **Conserver une ressource à la suppression de stack** : DeletionPolicy: Retain
- **Déploiement multi-comptes CFN** : StackSets
- **CDK : commande qui génère le template** : cdk synth
- **Verrouiller le state Terraform** : Backend S3 + DynamoDB (ou lock S3)
- **Fichier de build CodeBuild** : buildspec.yml
- **Fichier de déploiement CodeDeploy EC2** : appspec.yml
- **Blue/green vs canary** : Bascule complète vs progressive
- **Build once, deploy many** : Même artefact promu sur tous les environnements

Observabilité et sécurité

- **Qui a appelé l'API** : CloudTrail
- **Historique/conformité de configuration** : AWS Config
- **Métriques/alarmes** : CloudWatch
- **Mémoire EC2 dans CloudWatch** : Agent CloudWatch
- **Traçage distribué** : X-Ray / OpenTelemetry
- **Requêtes sur les logs** : CloudWatch Logs Insights
- **Détection de menaces** : GuardDuty
- **Vulnérabilités EC2/ECR/Lambda** : Inspector
- **Données sensibles dans S3** : Macie
- **Agrégation sécurité/conformité** : Security Hub
- **Chiffrement de grandes données avec KMS** : Envelope encryption (GenerateDataKey)
- **Rotation auto de mot de passe BD** : Secrets Manager

- **Config sensible gratuite** : SSM Parameter Store (SecureString)
- **Garde-fous multi-comptes** : SCP via Organizations / Control Tower
- **Landing zone** : AWS Control Tower

Résilience et coûts

- **RPO / RTO** : Perte de données tolérée / durée d'indisponibilité tolérée
- **Stratégies DR (du moins au plus cher)** : Backup&Restore, Pilot Light, Warm Standby, Multi-site actif-actif
- **Tester la résilience** : AWS Fault Injection Service
- **Recommandation de taille d'instances** : Compute Optimizer
- **Détecter les dépenses anormales** : Cost Anomaly Detection
- **Attribuer les coûts** : Tags d'allocation de coûts
- **Premier levier FinOps** : Visibilité (tags, budgets) puis suppression du gaspillage

Aide-mémoire pour l'examen

1. Mot-clé → service

Dans l'énoncé…	Pensez à…
« sans gérer de serveurs », « événementiel »	Lambda, Fargate, API Gateway, DynamoDB
« découpler », « tampon », « pic de charge »	SQS (+ SNS fan-out)
« ordre garanti », « exactly-once »	SQS FIFO
« diffuser à plusieurs abonnés »	SNS / EventBridge
« flux temps réel », « rejouer », « analytique en streaming »	Kinesis Data Streams / Firehose
« workflow », « étapes », « attente humaine »	Step Functions
« latence en microsecondes »	DAX (DynamoDB), ElastiCache
« IP statique », « TCP/UDP », « très faible latence »	NLB / Global Accelerator
« routage par chemin/hôte », « HTTP »	ALB
« cache mondial », « contenu statique », « réduire la latence »	CloudFront
« basculer automatiquement vers un site de secours »	Route 53 failover + health checks
« DDoS »	Shield (+ WAF, CloudFront)
« injection SQL », « XSS », « limitation de débit »	WAF
« menaces », « comportement anormal »	GuardDuty
« vulnérabilités logicielles »	Inspector
« données sensibles / PII dans S3 »	Macie
« qui a fait cet appel API »	CloudTrail
« configuration conforme / historique »	AWS Config
« métrique, alarme, dashboard »	CloudWatch
« traçage de requêtes », « goulot entre services »	X-Ray
« clés de chiffrement, audit »	KMS
« rotation automatique des identifiants »	Secrets Manager
« configuration applicative centralisée »	SSM Parameter Store / AppConfig
« accès au serveur sans SSH », « patcher à grande échelle »	Systems Manager

Dans l'énoncé…	Pensez à…
« infrastructure en code, déclaratif »	CloudFormation / CDK / Terraform
« multi-comptes, garde-fous »	Organizations + SCP, Control Tower
« partager des ressources entre comptes »	RAM
« migrer une base avec peu d'arrêt »	DMS
« migrer des serveurs »	Application Migration Service (MGN)
« transférer de gros volumes en hors-ligne »	Snow Family
« transfert en ligne vers S3/EFS/FSx »	DataSync
« data lake / SQL sur S3 sans serveur »	Athena + Glue
« conteneurs, pas de Kubernetes »	ECS (Fargate)
« Kubernetes managé »	EKS
« PaaS web simple »	Elastic Beanstalk / App Runner
« moindre effort opérationnel »	service managé / serverless
« le plus économique »	Spot, S3 IA/Glacier, Savings Plans, serverless, Graviton
« haute disponibilité »	Multi-AZ, ELB, ASG
« reprise après sinistre »	Multi-région, Backup, Aurora Global, DRS

2. Limites et chiffres à connaître

Élément	Valeur
Lambda	15 min max, 10 Go de mémoire, 1 000 exécutions simultanées/région (défaut), paquet 50 Mo zip / 250 Mo
SQS	message ≤ 256 Ko ; rétention 1 min–14 j ; visibility timeout 0–12 h ; long polling ≤ 20 s
SNS	message ≤ 256 Ko
S3	objet ≤ 5 To ; PUT unique ≤ 5 Go ; multipart conseillé > 100 Mo ; 3 500 PUT / 5 500 GET par seconde et par préfixe
DynamoDB	item ≤ 400 Ko ; 1 RCU = 4 Ko fortement cohérent ; 1 WCU = 1 Ko
EBS	gp3 jusqu'à 16 000 IOPS ; io2 Block Express 256 000 IOPS ; volumes ≤ 64 Tio
VPC	5 VPC/région (défaut, ajustable) ; CIDR /16 à /28 ; 5 IP réservées/subnet
Security Groups	60 règles entrantes + 60 sortantes (défaut) ; 5 SG par ENI (défaut)
Route 53	health checks toutes les 10 ou 30 s

Élément	Valeur
CloudWatch	métriques 5 min (EC2 base), 1 min (détaillé), 1 s (haute résolution) ; métriques conservées 15 mois
CloudTrail	historique 90 jours (événements de gestion)
API Gateway	timeout d'intégration 29 s (défaut)
ALB	100 règles/listener (défaut) ; idle timeout 60 s
IAM	policy managée ≤ 6 144 caractères ; 10 policies managées/rôle (défaut) ; session de rôle 1-12 h
KMS	chiffrement direct ≤ 4 Ko
RDS	sauvegardes 1-35 jours ; 15 réplicas de lecture (Aurora et RDS MySQL/MariaDB/PostgreSQL)
Secrets Manager	secret ≤ 64 Ko
EFS	NFSv4.1, port 2049

Les quotas évoluent : confirmez dans **Service Quotas** et la documentation.

3. Comparatifs à savoir par cœur

Stockage de blocs/fichiers/objets

	EBS	EFS	S3
Type	Bloc	Fichier NFS	Objet
Portée	1 AZ	Multi-AZ (région)	Région (accès global)
Accès concurrent	1 instance (Multi-Attach io2)	Milliers	Illimité

Messaging

	SQS	SNS	EventBridge	Kinesis
Modèle	File (pull)	Pub/Sub (push)	Bus à règles	Flux partitionné
Rétention	≤ 14 j	non	archive/replay	1-365 j
Rejeu	non	non	oui (archive)	oui

Chiffrement S3

SSE-S3 (clés AWS, défaut) · SSE-KMS (clé client, audit) · SSE-C (clé fournie par le client à chaque requête) · côté client.

Déploiements

Stratégie	Interruption	Retour arrière	Coût
All-at-once	oui	lent	bas
Rolling	non (capacité réduite)	moyen	bas
Immutable / Blue-Green	non	rapide	élevé (doublé brièvement)
Canary	non	rapide	moyen

DR

Stratégie	RTO/RPO	Coût
Backup & Restore	heures	$
Pilot Light	dizaines de minutes	$$
Warm Standby	minutes	$$$
Actif-actif	~0	

4. Règles de décision AWS (heuristiques d'examen)

1. **Managé** > auto-géré ; **serverless** > serveurs ; **découplage** > appel direct.
2. **Rôle IAM** > clés d'accès ; **credentials temporaires** partout.
3. **Moindre privilège** + conditions.
4. **Multi-AZ** par défaut ; multi-région seulement si RTO/RPO ou conformité l'exigent.
5. **Chiffrer** au repos (KMS) et en transit (TLS).
6. **Automatiser** (IaC, CI/CD, remédiation événementielle).
7. Réseau privé : **endpoints** plutôt que Internet/NAT.
8. Coûts : choisir le **moins cher qui satisfait toutes les exigences** (pas le plus puissant).
9. Si deux réponses semblent bonnes, retenir celle qui demande **le moins d'effort opérationnel** (si la question le demande).
10. Méfiez-vous des réponses « manuelles », « script cron sur EC2 », « clé stockée ».

5. Pièges classiques

- Multi-AZ ≠ réplica de lecture.
- Un subnet = une AZ ; EBS = une AZ.
- SG stateful, NACL stateless.
- ACM pour CloudFront = us-east-1.
- Chiffrer une RDS/EBS existante non chiffrée = via snapshot + copie.
- CloudTrail (API) ≠ CloudWatch (perfs) ≠ Config (config).
- SCP ne s'applique pas au compte de gestion ; ne donne pas de droits.
- S3 : `ListBucket` sur le bucket, `GetObject` sur `bucket/*`.

- Lambda ne peut pas dépasser 15 min.
- SQS Standard peut livrer des doublons : idempotence.

Suivi de progression et cahier d'erreurs

1. Objectif

- Certification visée : _____ · Date d'examen : //___
- Heures/semaine disponibles : ___

2. Tableau de suivi des modules

Module	Cours lu	Démos vues	Labo réalisé	Quiz ≥ 90 %	Flashcards	Date
00 Démarrage	☐	☐	☐	☐	☐	
01 Fondamentaux	☐	☐	☐	☐	☐	
02 IAM	☐	☐	☐	☐	☐	
03 VPC	☐	☐	☐	☐	☐	
04 EC2	☐	☐	☐	☐	☐	
05 Stockage	☐	☐	☐	☐	☐	
06 Bases de données	☐	☐	☐	☐	☐	
07 HA & edge	☐	☐	☐	☐	☐	
08 Serverless	☐	☐	☐	☐	☐	
09 Conteneurs	☐	☐	☐	☐	☐	
10 IaC	☐	☐	☐	☐	☐	
11 CI/CD	☐	☐	☐	☐	☐	
12 Observabilité	☐	☐	☐	☐	☐	
13 Sécurité	☐	☐	☐	☐	☐	
14 Architecture & FinOps	☐	☐	☐	☐	☐	
15 Multi-comptes	☐	☐	☐	☐	☐	
16 Projet final	—	—	☐	—	—	

3. Examens blancs

#	Date	Source	Score	Durée	Domaine le plus faible
1					
2					

#	Date	Source	Score	Durée	Domaine le plus faible
3					

Critère de décision : **≥ 80 % sur 2 examens consécutifs** → réservez l'examen.

4. Cahier d'erreurs

Pour chaque erreur, écrivez la **règle** qui aurait donné la bonne réponse.

Date	Question (résumé)	Ma réponse	Bonne réponse	Concept manquant / règle de décision

5. Plan de la dernière semaine

Jour	Activité
J-7	Examen blanc complet, analyse
J-6	Revue des domaines faibles + labos associés
J-5	Flashcards + cheatsheet
J-4	Second examen blanc
J-3	Cahier d'erreurs, whitepaper Well-Architected
J-2	Flashcards légères, schémas de mémoire
J-1	Repos, vérifier l'identifiant, l'heure, le matériel (examen en ligne : test système)
Jour J	Sommeil, repas, arriver tôt, gestion du temps